train_models

August 28, 2024

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

[2]:
```
#my local path
df = pd.read_excel("/home/houssam/internship project/Ammonia production final␣
 ↪data.xlsx")
```

[ ]:
```
df.head()
```

[3]:
```
#creat data

data = {
    "NG FEED - Comp Mass Flow (KJ/h)": df.loc[2:, 'Unnamed: 3'],
    "Ratio (S/C)": df.loc[2:, 'Unnamed: 4'],
    "Mixed Steam- Comp Mass Flow (KJ/h)": df.loc[2:, 'Unnamed: 5'],
    "Reformer inlet temperature (°C)": df.loc[2:, 'Unnamed: 6'],
    "Reformer inlet pressure (kPa)": df.loc[2:, 'Unnamed: 7'],
    "Reformer outlet temperature (°C)": df.loc[2:, 'Unnamed: 8'],
    "Reformer outlet pressure (kPa)": df.loc[2:, 'Unnamed: 9'],
    "Reformer Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 10'],
    "HTS - Inlet temperature (°C)": df.loc[2:, 'Unnamed: 11'],
    "HTS - Inlet pressure (kPa)": df.loc[2:, 'Unnamed: 12'],
    "HTS - Outlet temperature (°C)": df.loc[2:, 'Unnamed: 13'],
    "HTS - Outlet pressure (kPa)": df.loc[2:, 'Unnamed: 14'],
    "E-HTS - Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 15'],
    "LTS - Inlet temperature (°C)": df.loc[2:, 'Unnamed: 16'],
    "LTS - Inlet pressure (kPa)": df.loc[2:, 'Unnamed: 17'],
    "LTS - Outlet temperature (°C)": df.loc[2:, 'Unnamed: 18'],
    "LTS - Outlet pressure (kPa)": df.loc[2:, 'Unnamed: 19'],
    "E-LTS - Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 20'],
    "Condensor inlet temperature (°C)": df.loc[2:, 'Unnamed: 21'],
    "Condensor inlet pressure (kPa)": df.loc[2:, 'Unnamed: 22'],
    "Condensor outlet temperature (°C)": df.loc[2:, 'Unnamed: 23'],
    "Condensor outlet pressure (kPa)": df.loc[2:, 'Unnamed: 24'],
    "E-Condensor- Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 25'],
    "CO2 emissions-Comp Mass Flow (KJ/h)": df.loc[2:, 'Unnamed: 26'],
```

```
    "CO2 removal- Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 27'],
    "Methanator inlet temperature (°C)": df.loc[2:, 'Unnamed: 28'],
    "Methanator-Outlet temperature (°C)": df.loc[2:, 'Unnamed: 29'],
    "Methanator-Outlet pressure (kPa)": df.loc[2:, 'Unnamed: 30'],
    "Methanator-Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 31'],
    "Hydrogen inlet temperature (°C)": df.loc[2:, 'Unnamed: 32'],
    "Hydrogen inlet pressure (kPa)": df.loc[2:, 'Unnamed: 33'],
    "Nitrogen inlet temperature (°C)": df.loc[2:, 'Unnamed: 34'],
    "Nitrogen inlet pressure (kPa)": df.loc[2:, 'Unnamed: 35'],
    "S-1 stream temperature (°C)": df.loc[2:, 'Unnamed: 36'],
    "S-1 stream pressure (kPa)": df.loc[2:, 'Unnamed: 37'],
    "K-100 - Delta P (kPa)": df.loc[2:, 'Unnamed: 38'],
    "S-2 stream temperature (°C)": df.loc[2:, 'Unnamed: 39'],
    "S-2 stream pressure (kPa)": df.loc[2:, 'Unnamed: 40'],
    "S-3 stream temperature (°C)": df.loc[2:, 'Unnamed: 41'],
    "S-3 stream pressure (kPa)": df.loc[2:, 'Unnamed: 42'],
    "K-102 - Delta P (kPa)": df.loc[2:, 'Unnamed: 43'],
    "S-4 stream pressure (kPa)": df.loc[2:, 'Unnamed: 44'],
    "Synthesis gas inlet temperature (°C)": df.loc[2:, 'Unnamed: 45'],
    "Synthesis gas inlet pressure (kPa)": df.loc[2:, 'Unnamed: 46'],
    "Synthesis gas outlet temperature (°C)": df.loc[2:, 'Unnamed: 47'],
    "Synthesis gas outlet pressure (kPa)": df.loc[2:, 'Unnamed: 48'],
    "H.B reactor Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 49'],
    "Separator inlet temperature (°C)": df.loc[2:, 'Unnamed: 50'],
    "Separator inlet pressure (kPa)": df.loc[2:, 'Unnamed: 51'],
    "Separator outlet pressure (kPa)": df.loc[2:, 'Unnamed: 52'],
    "E-separator- Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 53'],
    "Q-100 - Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 54'],
    "Q-102 - Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 55'],
    "Q-103 - Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 56'],
    "Q-104 - Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 57'],
    "Q-105- Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 58'],
    "E-1- Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 59'],
    "E-2- Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 60'],
    "E-3- Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 61'],
    "E-4- Energy consumption (KJ/h)": df.loc[2:, 'Unnamed: 62'],
    "Ammonia - Comp Mass Fraction": df.loc[2:, 'Unnamed: 63'],
    "Ammonia - Comp Mass Flow (kg/h)": df.loc[2:, 'Unnamed: 64']
}
df = pd.DataFrame(data)
```

[4]:
```
df = df.drop_duplicates()
df.shape
```

[4]: (32001, 62)

[12]:
```
# Remove the cases where the output of ammonia is out of this range [3,2(T/
↪day), 4,8(T/day)]
df = df[(df['Ammonia - Comp Mass Flow (kg/h)'] < 200) & (df['Ammonia - Comp␣
↪Mass Flow (kg/h)'] > 134)]
df.shape
```

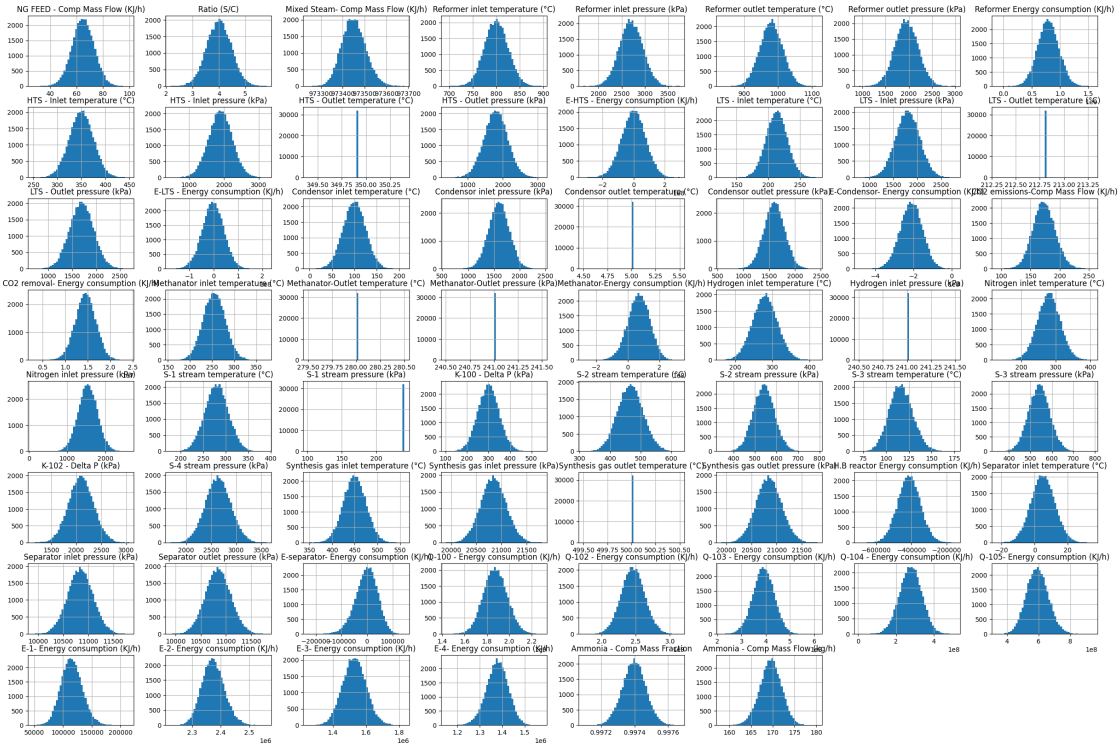
[12]:
```
(32001, 62)
```

[4]:
```
# Convert all columns to float
df = df.apply(pd.to_numeric, errors='coerce')
```

[45]:
```
df.hist(figsize=(30,20), bins=50)
```

[45]:
```
array([[<Axes: title={'center': 'NG FEED - Comp Mass Flow (KJ/h)'}>,
        <Axes: title={'center': 'Ratio (S/C)'}>,
        <Axes: title={'center': 'Mixed Steam- Comp Mass Flow (KJ/h)'}>,
        <Axes: title={'center': 'Reformer inlet temperature (°C)'}>,
        <Axes: title={'center': 'Reformer inlet pressure (kPa)'}>,
        <Axes: title={'center': 'Reformer outlet temperature (°C)'}>,
        <Axes: title={'center': 'Reformer outlet pressure (kPa)'}>,
        <Axes: title={'center': 'Reformer Energy consumption (KJ/h)'}>],
       [<Axes: title={'center': 'HTS - Inlet temperature (°C)'}>,
        <Axes: title={'center': 'HTS - Inlet pressure (kPa)'}>,
        <Axes: title={'center': 'HTS - Outlet temperature (°C)'}>,
        <Axes: title={'center': 'HTS - Outlet pressure (kPa)'}>,
        <Axes: title={'center': 'E-HTS - Energy consumption (KJ/h)'}>,
        <Axes: title={'center': 'LTS - Inlet temperature (°C)'}>,
        <Axes: title={'center': 'LTS - Inlet pressure (kPa)'}>,
        <Axes: title={'center': 'LTS - Outlet temperature (°C)'}>],
       [<Axes: title={'center': 'LTS - Outlet pressure (kPa)'}>,
        <Axes: title={'center': 'E-LTS - Energy consumption (KJ/h)'}>,
        <Axes: title={'center': 'Condensor inlet temperature (°C)'}>,
        <Axes: title={'center': 'Condensor inlet pressure (kPa)'}>,
        <Axes: title={'center': 'Condensor outlet temperature (°C)'}>,
        <Axes: title={'center': 'Condensor outlet pressure (kPa)'}>,
        <Axes: title={'center': 'E-Condensor- Energy consumption (KJ/h)'}>,
        <Axes: title={'center': 'CO2 emissions-Comp Mass Flow (KJ/h)'}>],
       [<Axes: title={'center': 'CO2 removal- Energy consumption (KJ/h)'}>,
        <Axes: title={'center': 'Methanator inlet temperature (°C)'}>,
        <Axes: title={'center': 'Methanator-Outlet temperature (°C)'}>,
        <Axes: title={'center': 'Methanator-Outlet pressure (kPa)'}>,
        <Axes: title={'center': 'Methanator-Energy consumption (KJ/h)'}>,
        <Axes: title={'center': 'Hydrogen inlet temperature (°C)'}>,
        <Axes: title={'center': 'Hydrogen inlet pressure (kPa)'}>,
        <Axes: title={'center': 'Nitrogen inlet temperature (°C)'}>],
       [<Axes: title={'center': 'Nitrogen inlet pressure (kPa)'}>,
        <Axes: title={'center': 'S-1 stream temperature (°C)'}>,
```

```
 <Axes: title={'center': 'S-1 stream pressure (kPa)'}>,
 <Axes: title={'center': 'K-100 - Delta P (kPa)'}>,
 <Axes: title={'center': 'S-2 stream temperature (°C)'}>,
 <Axes: title={'center': 'S-2 stream pressure (kPa)'}>,
 <Axes: title={'center': 'S-3 stream temperature (°C)'}>,
 <Axes: title={'center': 'S-3 stream pressure (kPa)'}>],
[<Axes: title={'center': 'K-102 - Delta P (kPa)'}>,
 <Axes: title={'center': 'S-4 stream pressure (kPa)'}>,
 <Axes: title={'center': 'Synthesis gas inlet temperature (°C)'}>,
 <Axes: title={'center': 'Synthesis gas inlet pressure (kPa)'}>,
 <Axes: title={'center': 'Synthesis gas outlet temperature (°C)'}>,
 <Axes: title={'center': 'Synthesis gas outlet pressure (kPa)'}>,
 <Axes: title={'center': 'H.B reactor Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'Separator inlet temperature (°C)'}>],
[<Axes: title={'center': 'Separator inlet pressure (kPa)'}>,
 <Axes: title={'center': 'Separator outlet pressure (kPa)'}>,
 <Axes: title={'center': 'E-separator- Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'Q-100 - Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'Q-102 - Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'Q-103 - Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'Q-104 - Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'Q-105- Energy consumption (KJ/h)'}>],
[<Axes: title={'center': 'E-1- Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'E-2- Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'E-3- Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'E-4- Energy consumption (KJ/h)'}>,
 <Axes: title={'center': 'Ammonia - Comp Mass Fraction'}>,
 <Axes: title={'center': 'Ammonia - Comp Mass Flow (kg/h)'}>,
 <Axes: >, <Axes: >]], dtype=object)
```



[7]:
```
# Check if all columns have only one unique value
all_unique_values = df.apply(lambda col: len(col.unique()) == 1)

unique_value_columns = all_unique_values[all_unique_values].index.tolist()
print("Columns with only one unique value:", unique_value_columns)
```

```
Columns with only one unique value: ['HTS - Outlet temperature (°C)', 'LTS -
Outlet temperature (°C)', 'Condensor outlet temperature (°C)', 'Methanator-
Outlet temperature (°C)', 'Methanator-Outlet pressure (kPa)', 'Hydrogen inlet
pressure (kPa)', 'Synthesis gas outlet temperature (°C)']
```

[12]:
```
df['S-1 stream pressure (kPa)'].unique()
```

[12]:
```
array([101, 241])
```

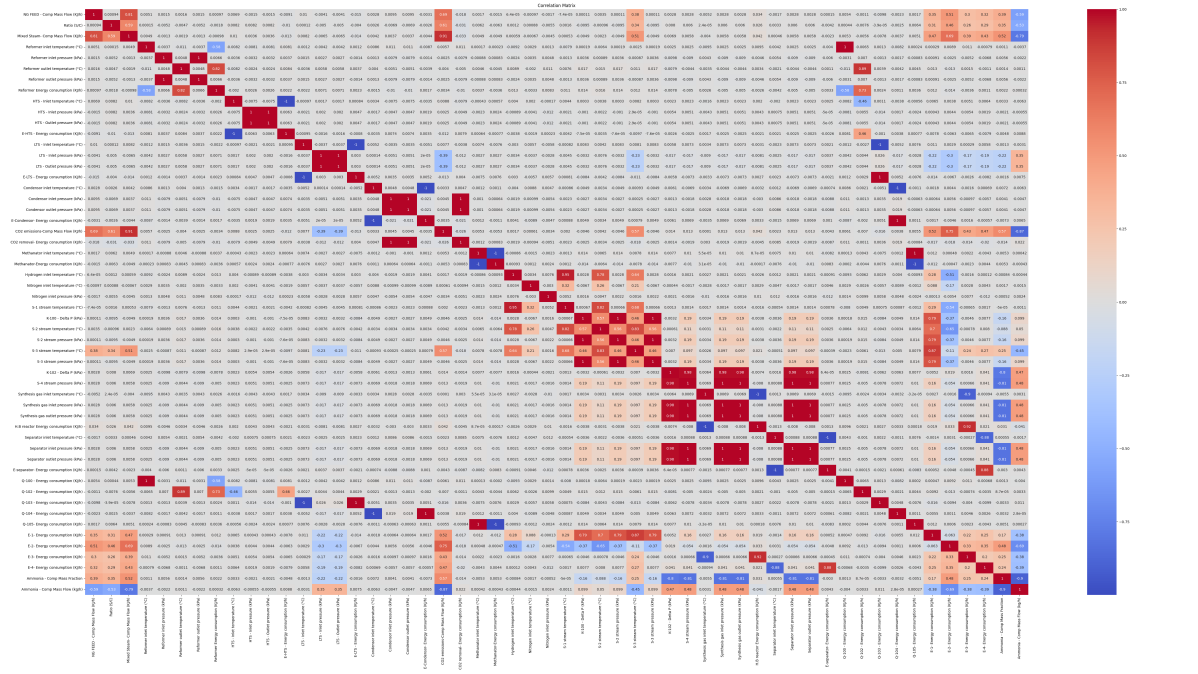
['S-1 stream pressure (kPa)','HTS - Outlet temperature (°C)', 'LTS - Outlet temperature (°C)', 'Condensor outlet temperature (°C)', 'Methanator-Outlet temperature (°C)', 'Methanator-Outlet pressure (kPa)', 'Hydrogen inlet pressure (kPa)', 'Synthesis gas outlet temperature (°C)']. ALL these columns are fixe so the won't have any impact on the output

[5]:
```

```

```
df.drop(columns=['S-1 stream pressure (kPa)','HTS - Outlet temperature (°C)',␣
 ↪'LTS - Outlet temperature (°C)', 'Condensor outlet temperature (°C)',␣
 ↪'Methanator-Outlet temperature (°C)', 'Methanator-Outlet pressure (kPa)',␣
 ↪'Hydrogen inlet pressure (kPa)', 'Synthesis gas outlet temperature (°C)'],␣
 ↪inplace=True)
```

0.1 Correlation matrix

[27]:
```
import seaborn as sns
def correlation_matrix(X):
    correlation_matrix = X.corr()
    plt.figure(figsize=(60, 30))
    sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
    plt.title('Correlation Matrix')
    plt.show()

correlation_matrix(df)
```



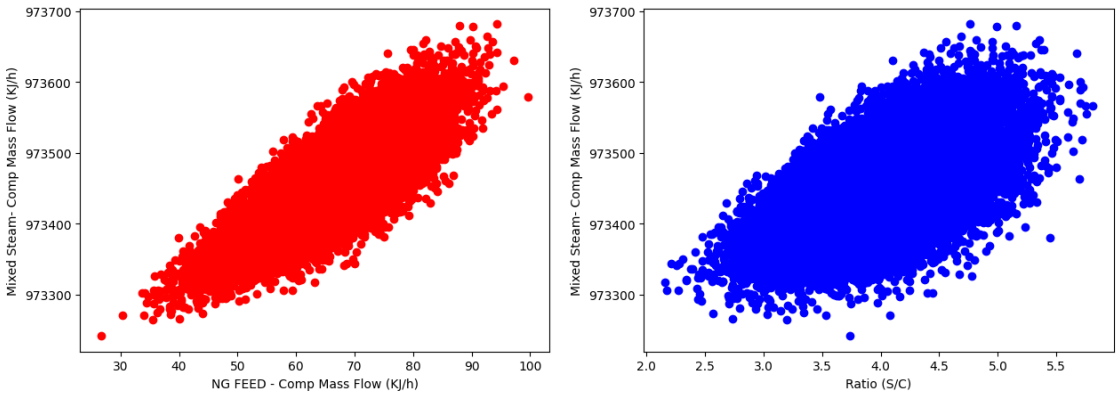
[28]:
```
def Sub_correlation_matrix(y, df):
    corr_matrix = df.corr()

    # We drop all the other energies and the ammonia mass Flow because the are␣
 ↪not known values
    drop_columns = [
```

```
        'E-separator- Energy consumption (KJ/h)', 'Q-100 - Energy consumption␣
↪(KJ/h)',
        'Q-102 - Energy consumption (KJ/h)', 'Q-103 - Energy consumption (KJ/
↪h)',
        'Q-104 - Energy consumption (KJ/h)', 'Q-105- Energy consumption (KJ/h)',
        'E-1- Energy consumption (KJ/h)', 'E-2- Energy consumption (KJ/h)',
        'E-3- Energy consumption (KJ/h)', 'E-4- Energy consumption (KJ/h)',
        'Ammonia - Comp Mass Fraction', 'Ammonia - Comp Mass Flow (kg/h)',
        'H.B reactor Energy consumption (KJ/h)', 'Methanator-Energy consumption␣
↪(KJ/h)',
        'CO2 emissions-Comp Mass Flow (KJ/h)', 'CO2 removal- Energy consumption␣
↪(KJ/h)',
        'E-Condensor- Energy consumption (KJ/h)', 'E-LTS - Energy consumption␣
↪(KJ/h)',
        'E-HTS - Energy consumption (KJ/h)', 'Reformer Energy consumption (KJ/
↪h)',
        'NG FEED - Comp Mass Flow (KJ/h)', 'Ratio (S/C)', y
    ]

    X = df.drop(columns=drop_columns)

    correlated_features = []
    for col in X.columns:
        if abs(corr_matrix[col][y]) >= 0.05:
            correlated_features.append(col)
        else:
            X.drop(columns=[col], inplace=True)

    plt.figure(figsize=(10, 2))
    sns.heatmap(corr_matrix.loc[[y] , correlated_features], annot=True,␣
↪cmap='coolwarm')
    plt.title('Correlation Matrix of Selected Features')
    plt.show()

    return X
```

0.1.1 First model that predict the 'Mixed Steam- Comp Mass Flow (KJ/h)'

[95]:
```
X = df[['NG FEED - Comp Mass Flow (KJ/h)', 'Ratio (S/C)']]
y = df['Mixed Steam- Comp Mass Flow (KJ/h)']
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].scatter(X['NG FEED - Comp Mass Flow (KJ/h)'], y, color='red')
axes[0].set_xlabel("NG FEED - Comp Mass Flow (KJ/h)")
axes[0].set_ylabel("Mixed Steam- Comp Mass Flow (KJ/h)")
```

```
axes[1].scatter(X['Ratio (S/C)'], y, color='blue')
axes[1].set_xlabel("Ratio (S/C)")
axes[1].set_ylabel("Mixed Steam- Comp Mass Flow (KJ/h)")

plt.show()
```



[14]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
import os
import joblib
```

[96]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('regression', LinearRegression()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))
```

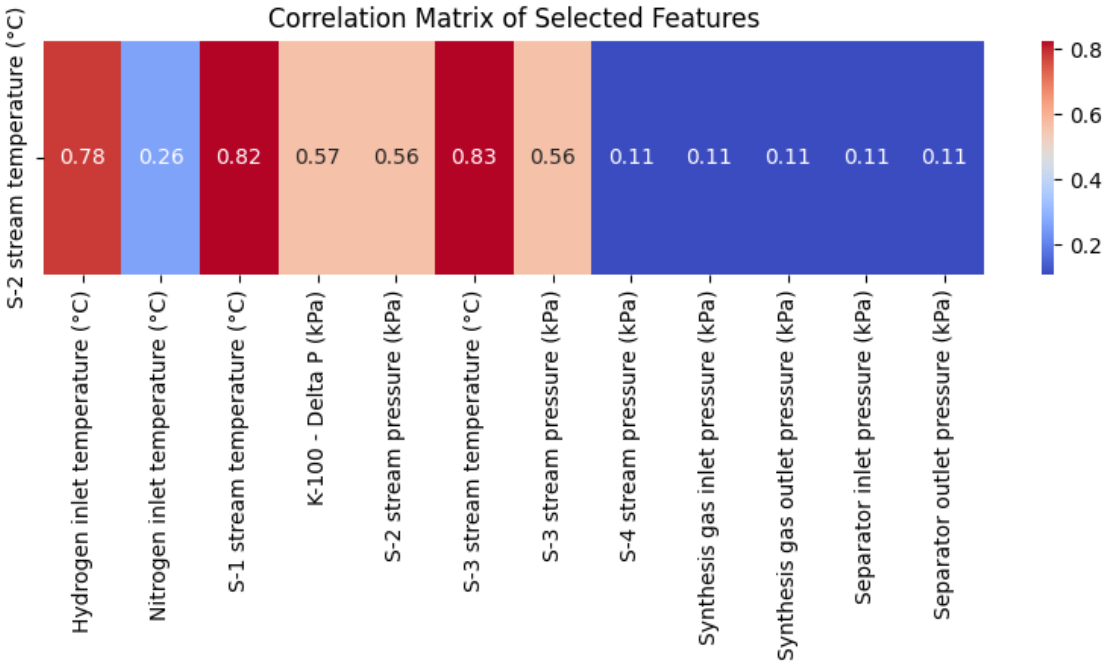
```
# Save the pipeline to a file
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/
 ↪Mixed_Steam_Pipeline.pkl')
```

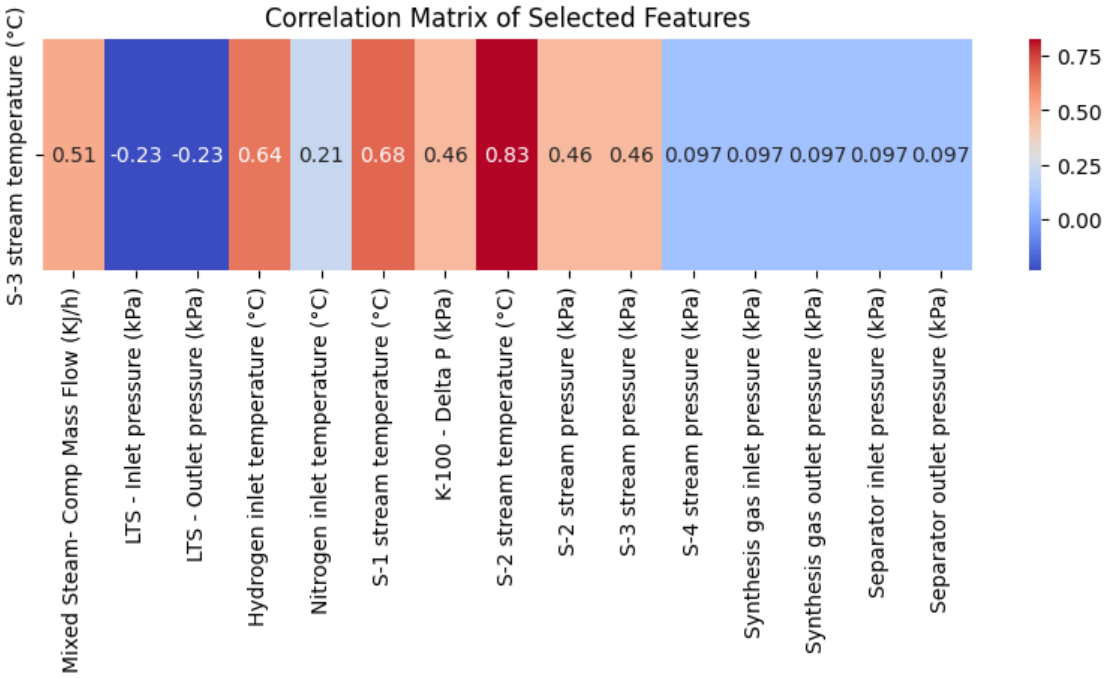
```
Mean Squared Error: 17.173786807853162
R^2 Score: 0.9943209856517062
```

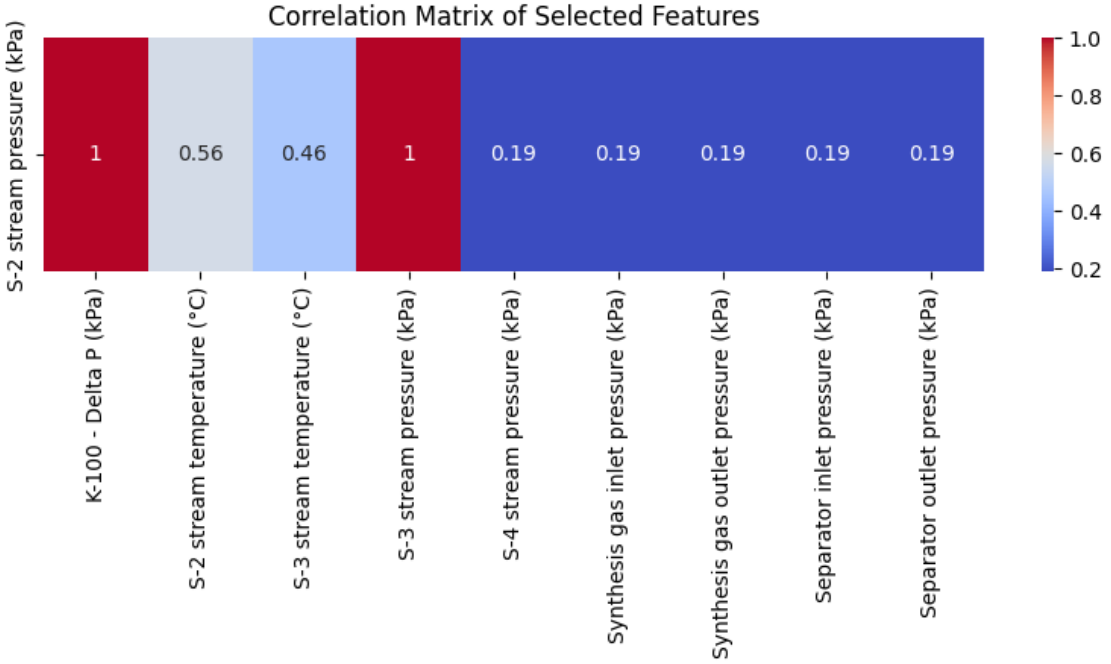
[96]:
```
['/home/houssam/internship project/trained_models/Mixed_Steam_Pipeline.pkl']
```

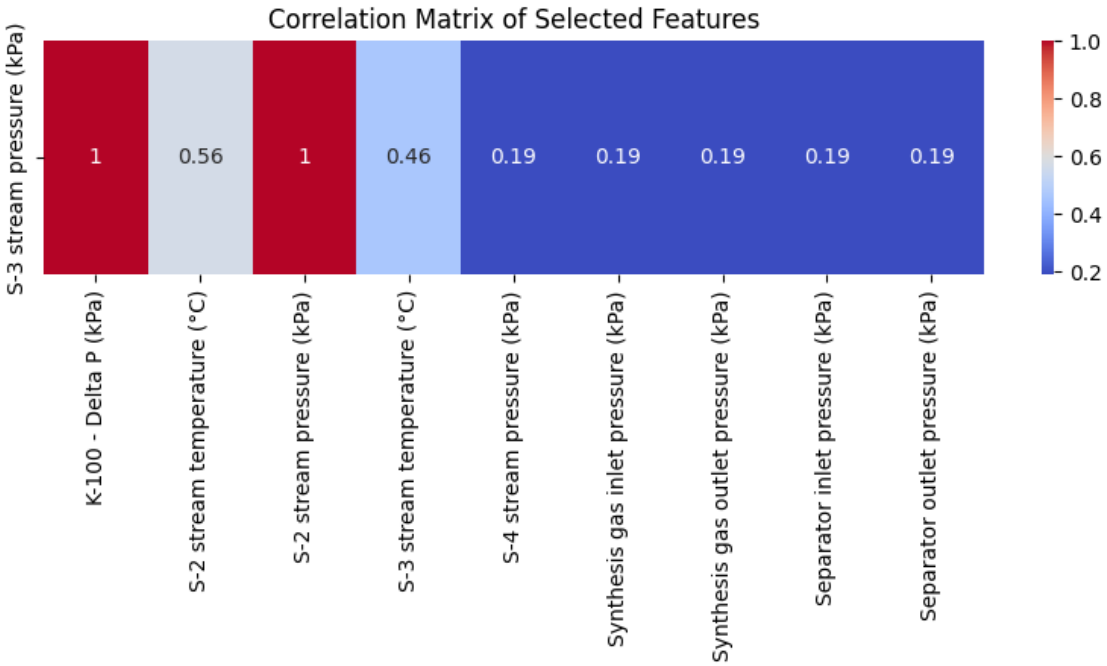
0.2 S stream models

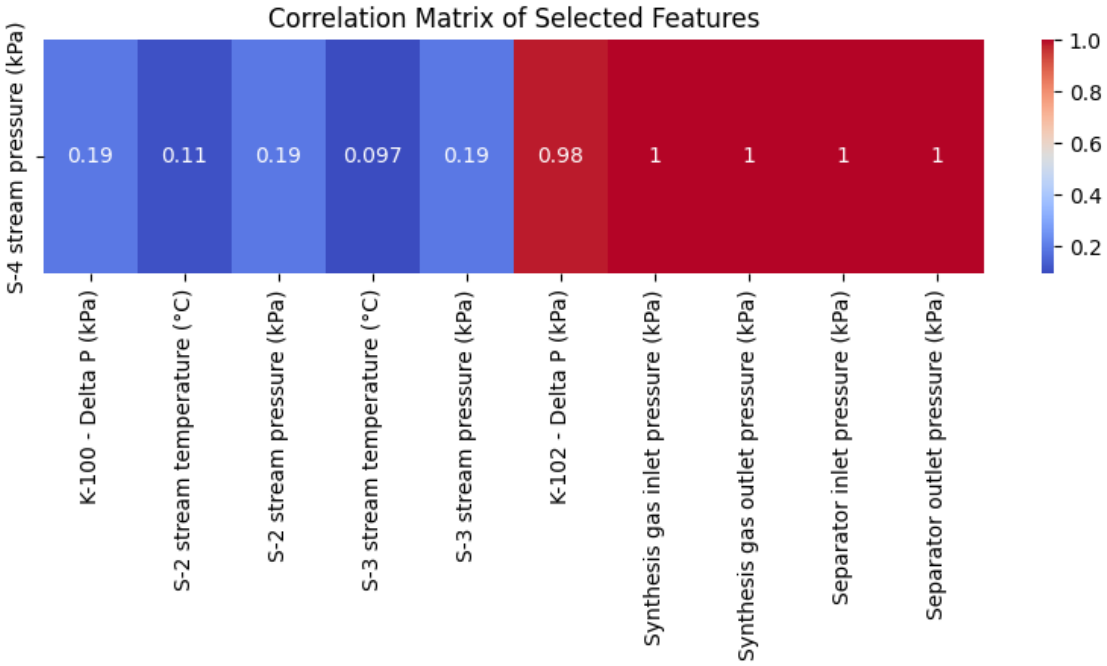
[29]:
```
X = Sub_correlation_matrix('S-2 stream temperature (°C)', df)
X = Sub_correlation_matrix('S-3 stream temperature (°C)', df)
X = Sub_correlation_matrix('S-2 stream pressure (kPa)', df)
X = Sub_correlation_matrix('S-3 stream pressure (kPa)', df)
X = Sub_correlation_matrix('S-4 stream pressure (kPa)', df)
```




Correlation Matrix of Selected Features
S-3 stream temperature (°C)
0.51
-0.23
-0.23
0.64
0.21
0.68
0.46
0.83
0.46
0.46
0.097
0.097
0.097
0.097
0.097
Mixed Steam- Comp Mass Flow (KJ/h)
LTS - Inlet pressure (kPa)
LTS - Outlet pressure (kPa)
Hydrogen inlet temperature (°C)
Nitrogen inlet temperature (°C)
S-1 stream temperature (°C)
K-100 - Delta P (kPa)
S-2 stream temperature (°C)
S-2 stream pressure (kPa)
S-3 stream pressure (kPa)
S-4 stream pressure (kPa)
Synthesis gas inlet pressure (kPa)
Synthesis gas outlet pressure (kPa)
Separator inlet pressure (kPa)
Separator outlet pressure (kPa)
0.75
0.50
0.25
0.00


Correlation Matrix of Selected Features
S-2 stream pressure (kPa)
1
0.56
0.46
1
0.19
0.19
0.19
0.19
0.19
K-100 - Delta P (kPa)
S-2 stream temperature (°C)
S-3 stream temperature (°C)
S-3 stream pressure (kPa)
S-4 stream pressure (kPa)
Synthesis gas inlet pressure (kPa)
Synthesis gas outlet pressure (kPa)
Separator inlet pressure (kPa)
Separator outlet pressure (kPa)
1.0
0.8
0.6
0.4
0.2





[97]:
```
from sklearn.ensemble import RandomForestRegressor
X = df[['Hydrogen inlet temperature (°C)', 'Nitrogen inlet temperature␣
 ↪(°C)','K-100 - Delta P (kPa)']]
```

```
y = df['S-2 stream temperature (°C)']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('RandomForest', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/S-2␣
 ↪stream temperature_Pipeline.pkl')
```

```
Mean Squared Error: 5.459386059643987
R^2 Score: 0.996865668722163
```

[97]:
```
['/home/houssam/internship project/trained_models/S-2 stream
temperature_Pipeline.pkl']
```

0.2.1 2. S-3 steam temperature (C°)

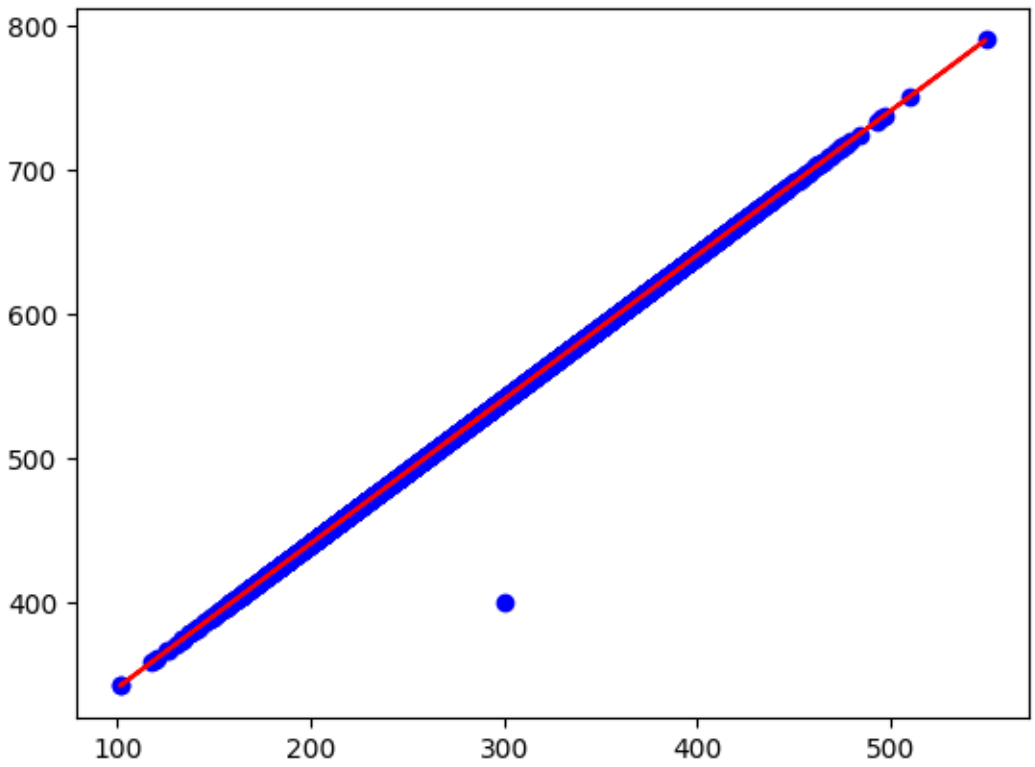
[ ]:
```
X = df[['Hydrogen inlet temperature (°C)', 'Nitrogen inlet temperature␣
 ↪(°C)','K-100 - Delta P (kPa)','Mixed Steam- Comp Mass Flow (KJ/h)','LTS -␣
 ↪Inlet pressure (kPa)']]
y = df['S-3 stream temperature (°C)']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('RandomForest', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)
```

```
# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/S-3␣
 ↪stream temperature_Pipeline.pkl')
```

0.2.2 3. S-2 steam pressure

[43]:
```
X = df['K-100 - Delta P (kPa)']
y = df['S-2 stream pressure (kPa)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

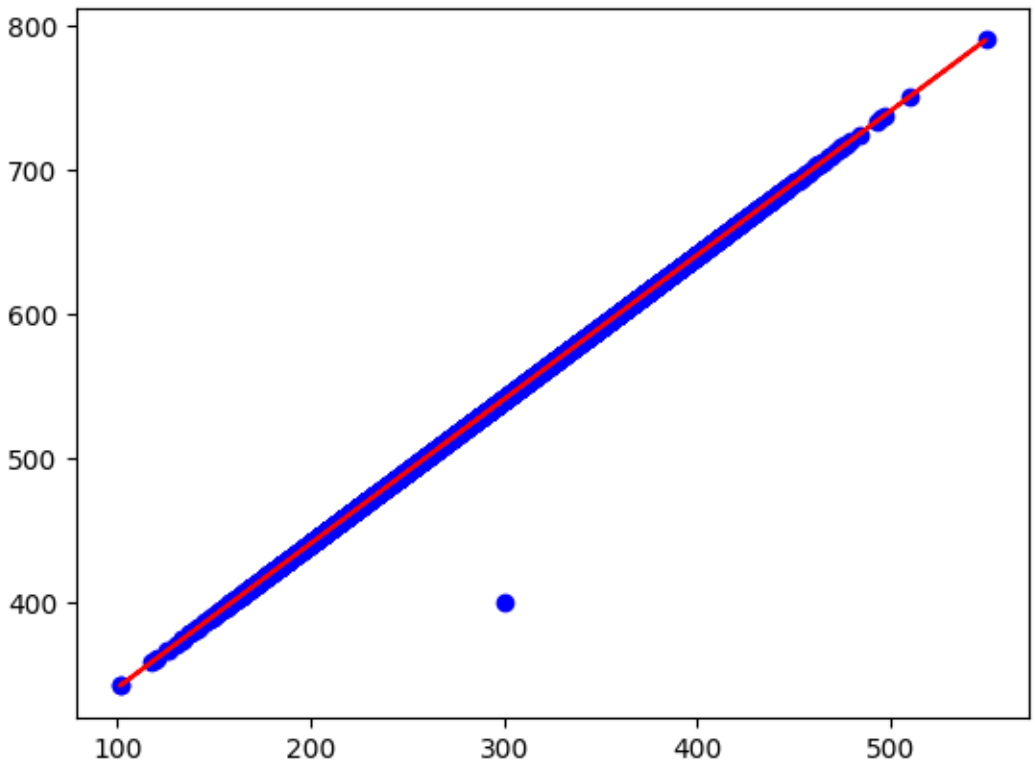
```
m:1.0000002102686611, c:240.99556203126969
```



0.2.3 4. S-3 steam pressure

[44]:
```
X = df[')']
y = df['S-3 stream pressure (kPa)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:1.0000002102686611, c:240.99556203126969
```



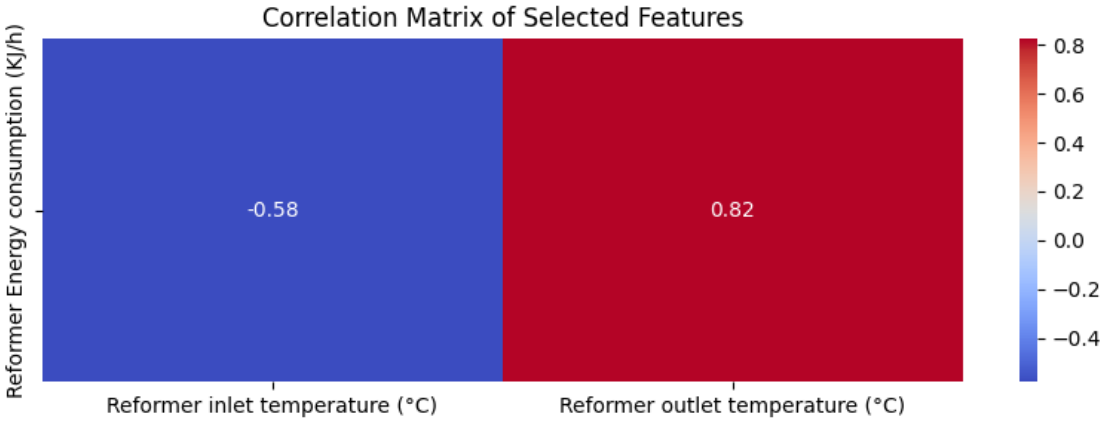
0.2.4 5. S-4 steam pressure

[ ]:
```
X = df['Synthesis gas inlet pressure (kPa)']
y = df['S-4 stream pressure (kPa)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

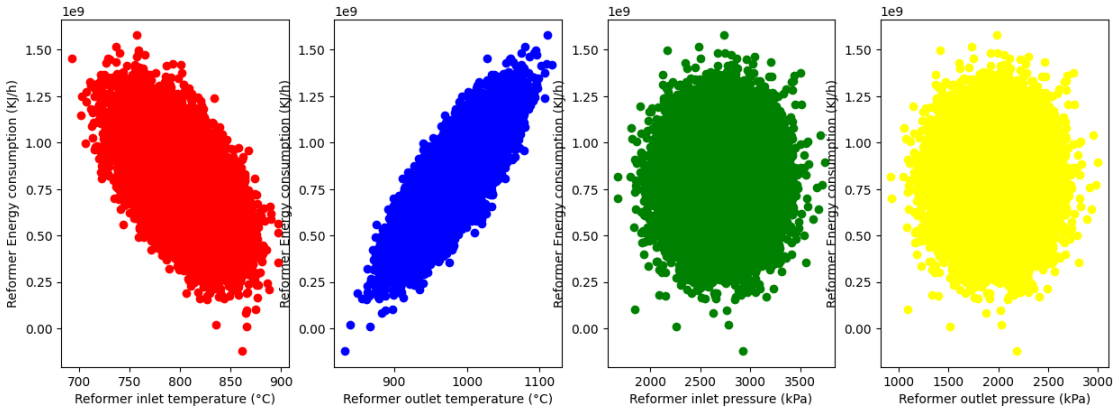
0.3 Energy consumption models

0.3.1 1. Reformer Energy consumption

[21]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('Reformer Energy consumption (KJ/h)', df)
```



[22]:
```
#Plot the data to choose the model
X = df[['Reformer inlet temperature (°C)','Reformer outlet temperature␣
 ↪(°C)','Reformer inlet pressure (kPa)','Reformer outlet pressure (kPa)']]
y = df['Reformer Energy consumption (KJ/h)']
fig, axes = plt.subplots(1, 4, figsize=(15, 5))

axes[0].scatter(X['Reformer inlet temperature (°C)'], y, color='red')
axes[0].set_xlabel("Reformer inlet temperature (°C)")
axes[0].set_ylabel("Reformer Energy consumption (KJ/h)")

axes[1].scatter(X['Reformer outlet temperature (°C)'], y, color='blue')
axes[1].set_xlabel("Reformer outlet temperature (°C)")
axes[1].set_ylabel("Reformer Energy consumption (KJ/h)")

axes[2].scatter(X['Reformer inlet pressure (kPa)'], y, color='green')
axes[2].set_xlabel("Reformer inlet pressure (kPa)")
axes[2].set_ylabel("Reformer Energy consumption (KJ/h)")

axes[3].scatter(X['Reformer outlet pressure (kPa)'], y, color='yellow')
axes[3].set_xlabel("Reformer outlet pressure (kPa)")
axes[3].set_ylabel("Reformer Energy consumption (KJ/h)")

plt.show()
```



[49]:
```
from sklearn.feature_selection import mutual_info_regression
def make_mi_scores(X,y):
    discrete_features = X.dtypes == int
    mi_scores = mutual_info_regression(X,y,discrete_features=discrete_features)
    mi_scores = pd.Series(mi_scores, name="MI Scores", index=X.columns)
    mi_scores = mi_scores.sort_values(ascending=False)
    return mi_scores

mi_scores = make_mi_scores(X, y)
mi_scores
```

[49]:
```
K-100 - Delta P (kPa)                0.520933
S-2 stream pressure (kPa)            0.520856
S-2 stream temperature (°C)          0.334082
Mixed Steam- Comp Mass Flow (KJ/h)   0.128967
Hydrogen inlet temperature (°C)      0.042351
LTS - Inlet pressure (kPa)           0.029743
Name: MI Scores, dtype: float64
```

The pressure doesn't effect the reformer energy consumption

[23]:
```
X = df[['Reformer inlet temperature (°C)','Reformer outlet temperature (°C)']]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('regression', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)
```

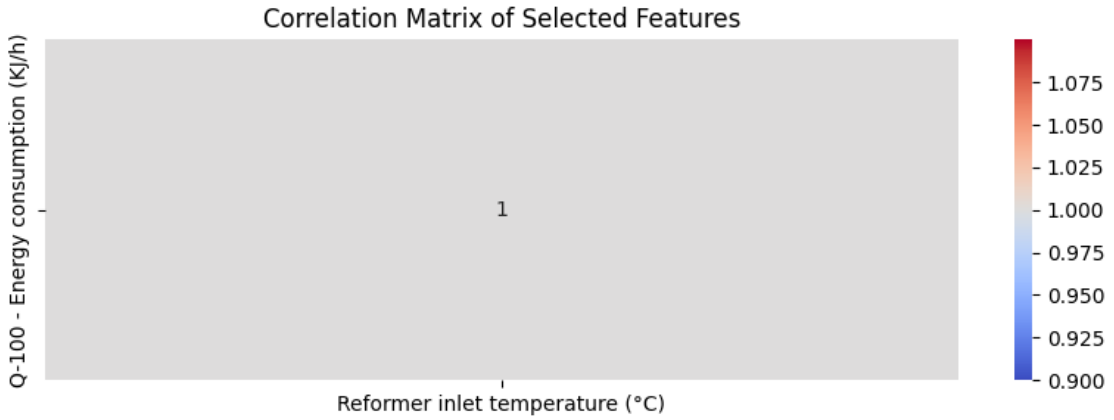
```
# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/
 ↪Reformer_Energy_Pipeline.pkl')
```

```
Mean Squared Error: 5519774496547.095
R^2 Score: 0.9998404720047122
```

[23]:
```
['/home/houssam/internship project/trained_models/Reformer_Energy_Pipeline.pkl']
```

0.3.2 2. Q-100-Energy consumption

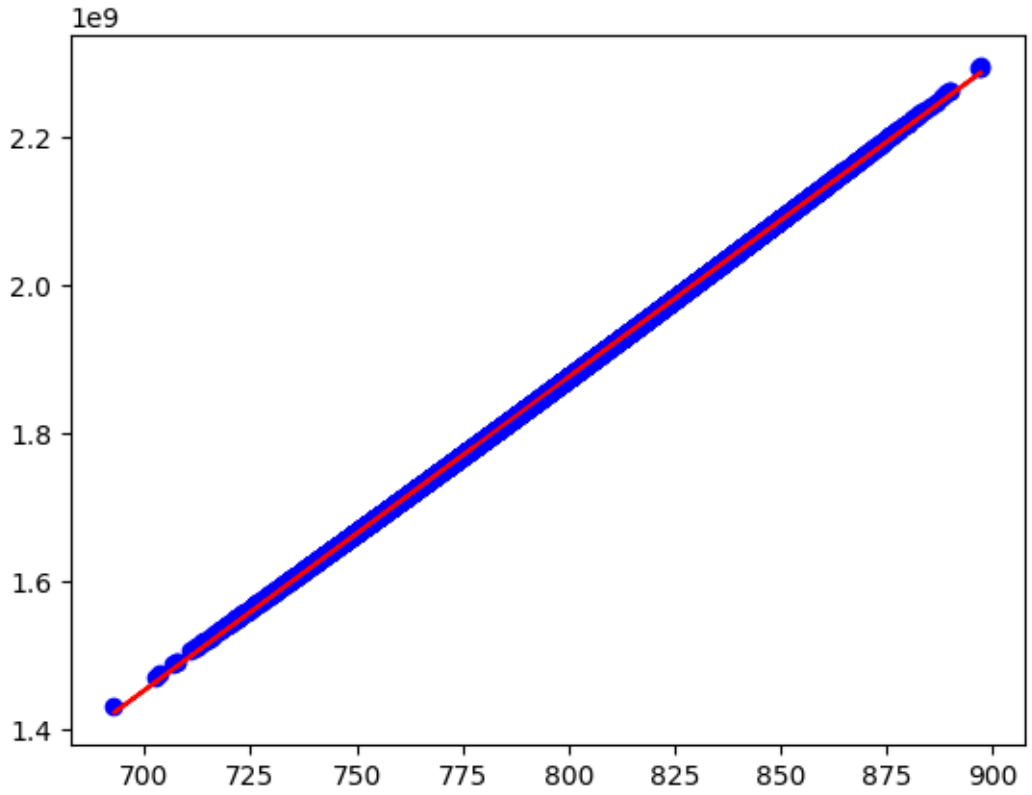
[25]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('Q-100 - Energy consumption (KJ/h)', df)
```



[27]:
```
X = df['Reformer inlet temperature (°C)']
y = df['Q-100 - Energy consumption (KJ/h)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)
```

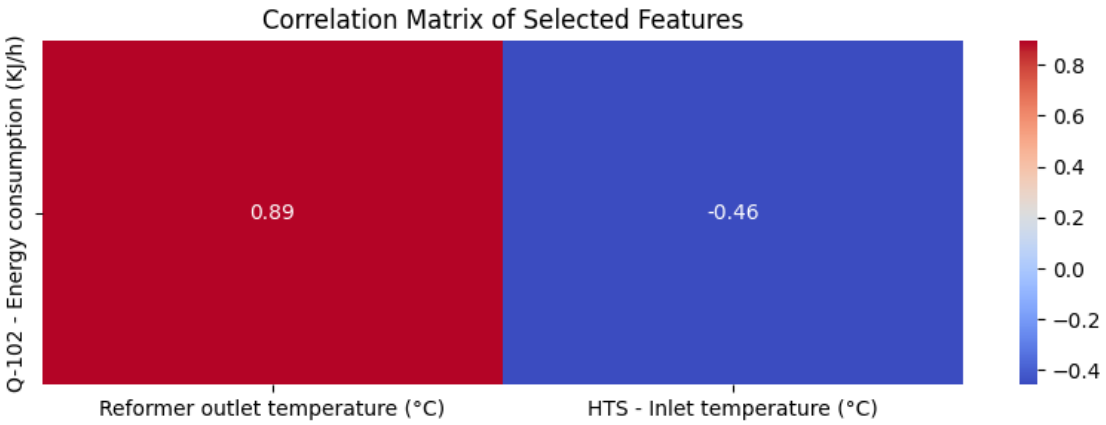
```
# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:4227654.693147199, c:-1507124898.3797724
```

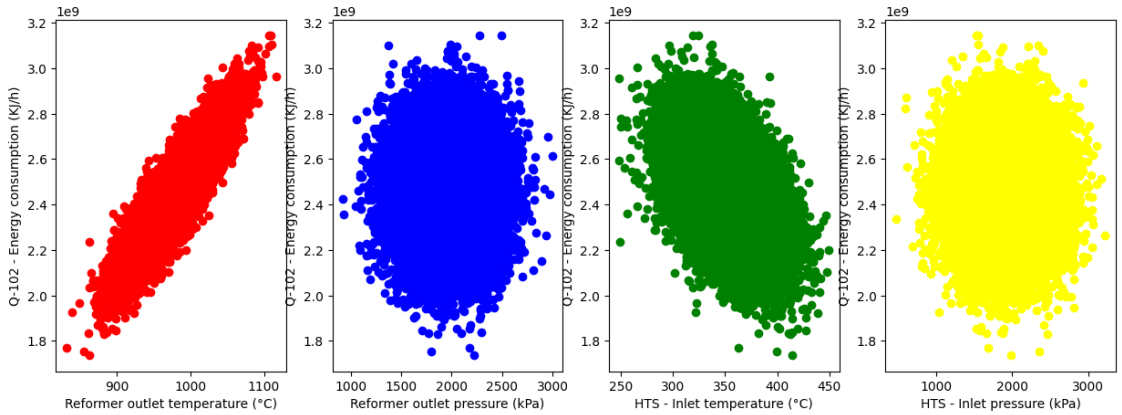


0.3.3 3. Q-102- Energy consumption

[28]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('Q-102 - Energy consumption (KJ/h)', df)
```



[20]:
```
#Plot the data to choose the model
X = df[['Reformer outlet temperature (°C)','Reformer outlet pressure␣
 ↪(kPa)','HTS - Inlet temperature (°C)','HTS - Inlet pressure (kPa)']]
y = df['Q-102 - Energy consumption (KJ/h)']
fig, axes = plt.subplots(1, 4, figsize=(15, 5))

axes[0].scatter(X['Reformer outlet temperature (°C)'], y, color='red')
axes[0].set_xlabel("Reformer outlet temperature (°C)")
axes[0].set_ylabel("Q-102 - Energy consumption (KJ/h)")

axes[1].scatter(X['Reformer outlet pressure (kPa)'], y, color='blue')
axes[1].set_xlabel("Reformer outlet pressure (kPa)")
axes[1].set_ylabel("Q-102 - Energy consumption (KJ/h)")

axes[2].scatter(X['HTS - Inlet temperature (°C)'], y, color='green')
axes[2].set_xlabel("HTS - Inlet temperature (°C)")
axes[2].set_ylabel("Q-102 - Energy consumption (KJ/h)")

axes[3].scatter(X['HTS - Inlet pressure (kPa)'], y, color='yellow')
axes[3].set_xlabel("HTS - Inlet pressure (kPa)")
axes[3].set_ylabel("Q-102 - Energy consumption (KJ/h)")

plt.show()
```



[31]:
```
mi_scores = make_mi_scores(X, y)
mi_scores
```

[31]:
```
Reformer outlet temperature (°C)    0.795238
HTS - Inlet temperature (°C)        0.106638
HTS - Inlet pressure (kPa)          0.001748
Reformer outlet pressure (kPa)      0.000000
Name: MI Scores, dtype: float64
```

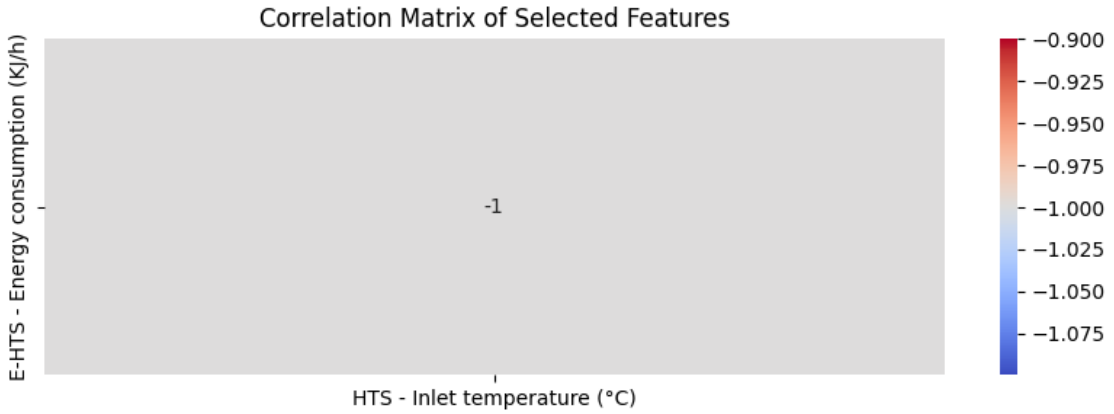
[21]:
```
X = df[['Reformer outlet temperature (°C)','HTS - Inlet temperature (°C)']]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('regression', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/Q-102 -␣
 ↪Energy_Pipeline.pkl')
```

```
Mean Squared Error: 7484373415217.656
R^2 Score: 0.9997496837255504
```

[21]:
```
['/home/houssam/internship project/trained_models/Q-102 - Energy_Pipeline.pkl']
```

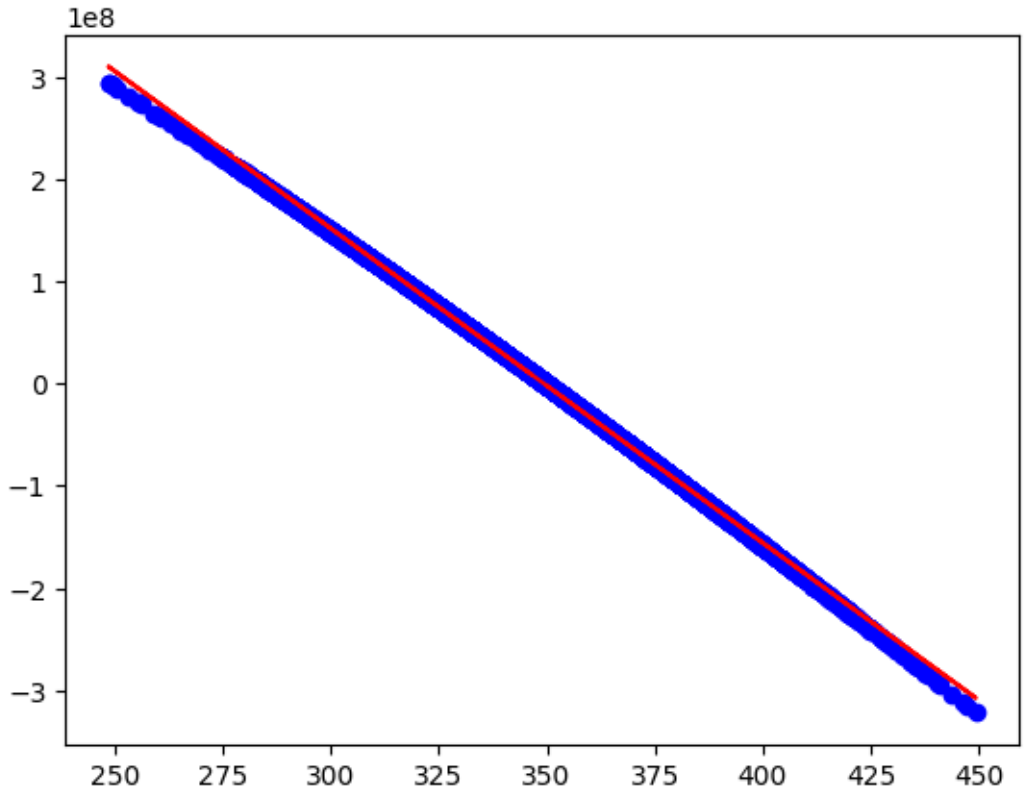
4. E-HTS - Energy consumption

[34]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('E-HTS - Energy consumption (KJ/h)', df)
```



[35]:
```
X = df['HTS - Inlet temperature (°C)']
y = df['E-HTS - Energy consumption (KJ/h)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:-3082293.5646714796, c:1077098983.4823322
```

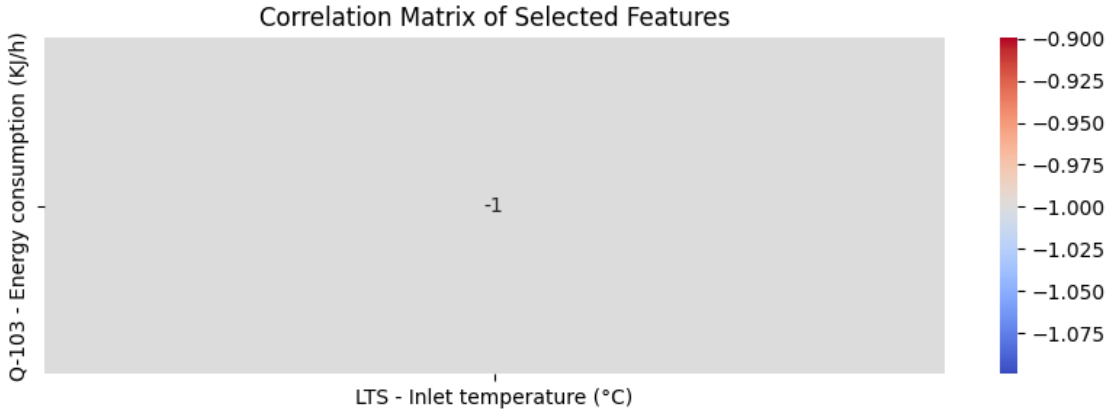


[14]:
```
-3082293.5646714796*349.4+1077098983.4823322
```

[14]: 145611.98611736298

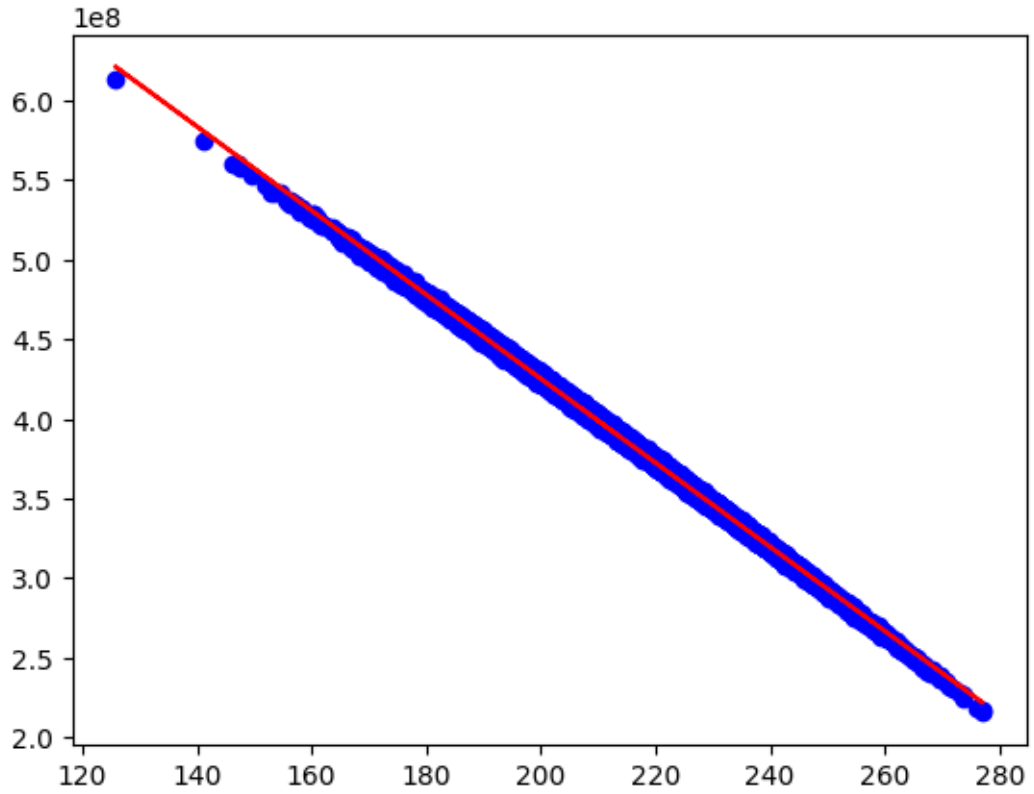
0.3.4 5. Q-103- Energy consumption

[42]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('Q-103 - Energy consumption (KJ/h)', df)
```



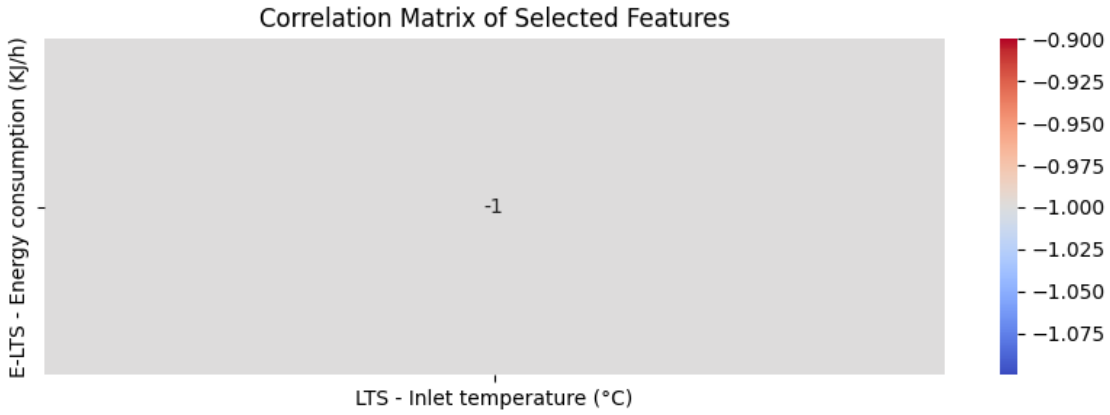
[40]:
```
X = df['LTS - Inlet temperature (°C)']
y = df['Q-103 - Energy consumption (KJ/h)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:-2640932.047069013, c:953513207.3016243
```



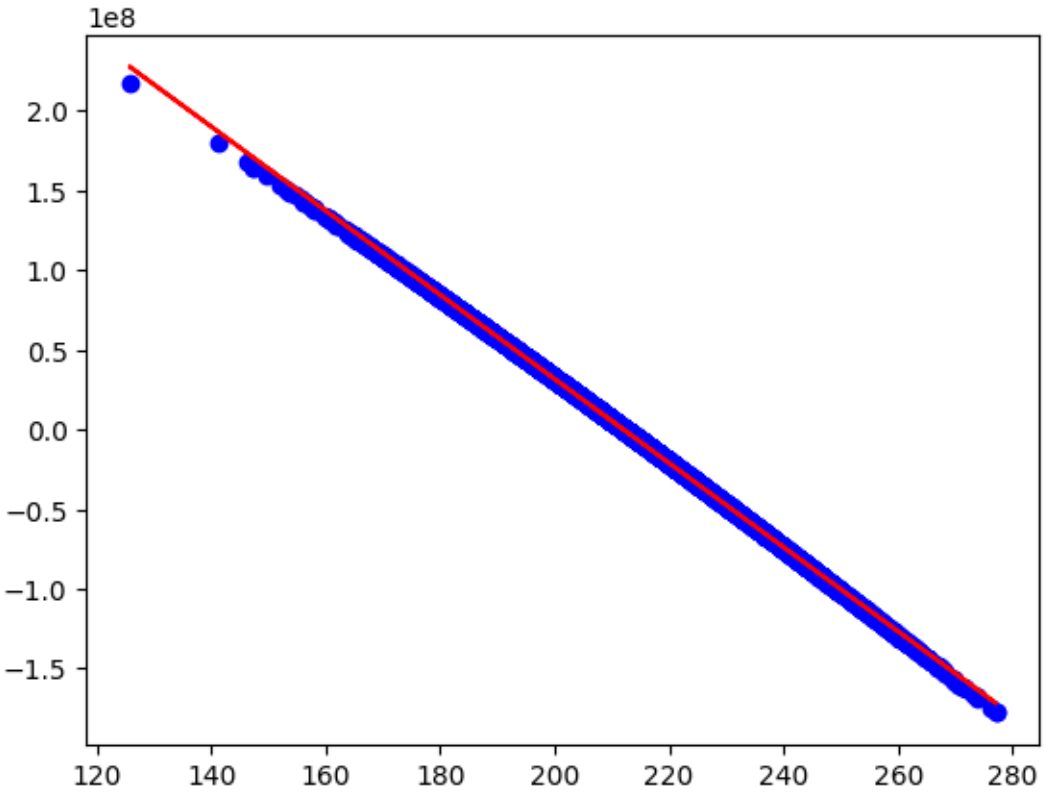
0.3.5 6. E-LTS - Energy consumption

[43]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('E-LTS - Energy consumption (KJ/h)', df)
```



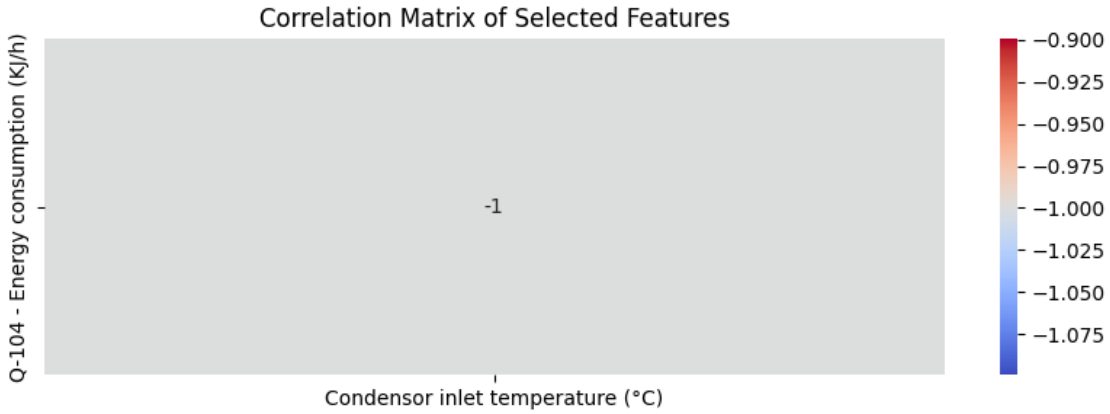
[41]:
```
X = df['LTS - Inlet temperature (°C)']
y = df['E-LTS - Energy consumption (KJ/h)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:-2640920.0637122733, c:559925580.9712701
```

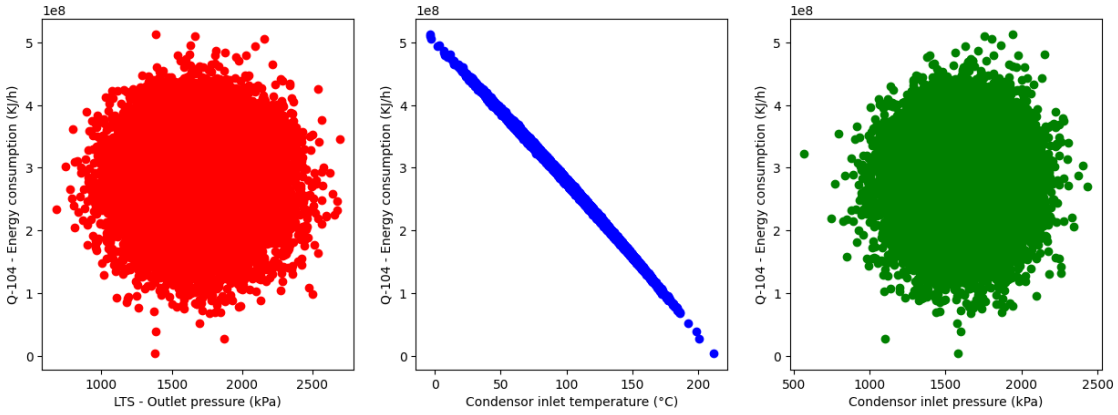


0.3.6 6. Q-104- Energy consumption

[44]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('Q-104 - Energy consumption (KJ/h)', df)
```



[45]:
```
#Plot the data to choose the model
X = df[['LTS - Outlet pressure (kPa)','Condensor inlet temperature␣
 ↪(°C)','Condensor inlet pressure (kPa)']]
y = df['Q-104 - Energy consumption (KJ/h)']
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].scatter(X['LTS - Outlet pressure (kPa)'], y, color='red')
axes[0].set_xlabel("LTS - Outlet pressure (kPa)")
axes[0].set_ylabel("Q-104 - Energy consumption (KJ/h)")

axes[1].scatter(X['Condensor inlet temperature (°C)'], y, color='blue')
axes[1].set_xlabel("Condensor inlet temperature (°C)")
axes[1].set_ylabel("Q-104 - Energy consumption (KJ/h)")

axes[2].scatter(X['Condensor inlet pressure (kPa)'], y, color='green')
axes[2].set_xlabel("Condensor inlet pressure (kPa)")
axes[2].set_ylabel("Q-104 - Energy consumption (KJ/h)")


plt.show()
```



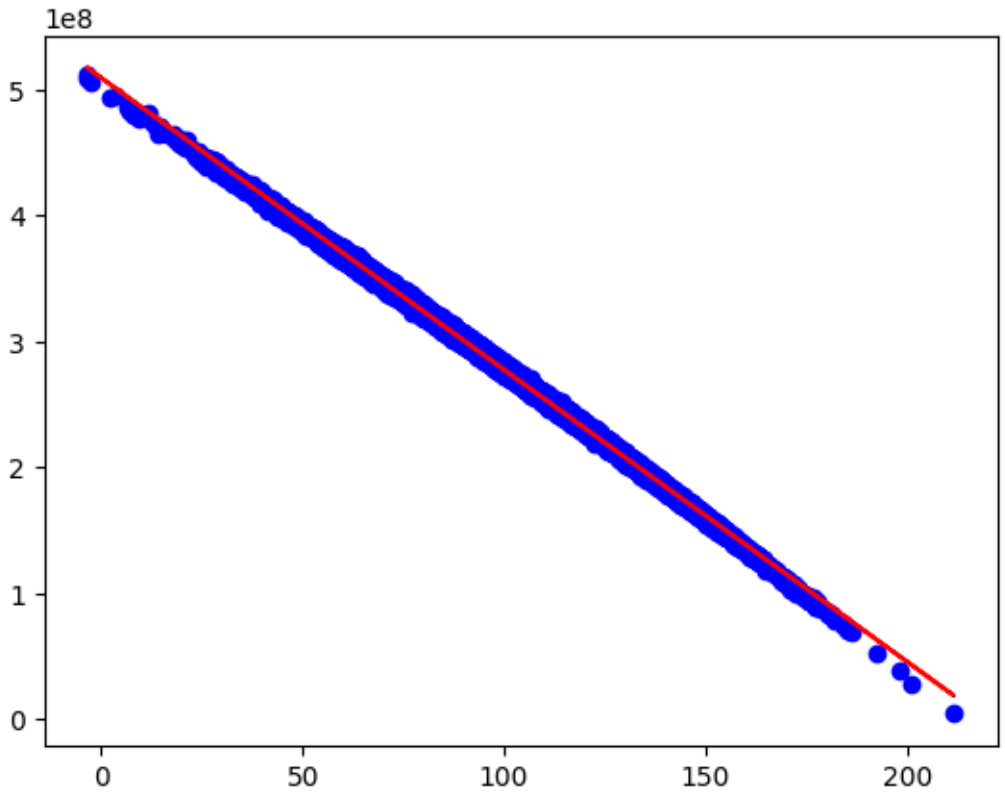
[43]:
```
mi_scores = make_mi_scores(X, y)
mi_scores
```

[43]:
```
Condensor inlet temperature (°C)    3.547086
Condensor inlet pressure (kPa)      0.001015
LTS - Outlet pressure (kPa)         0.000555
Name: MI Scores, dtype: float64
```

[46]:
```
X = df['Condensor inlet temperature (°C)']
y = df['Q-104 - Energy consumption (KJ/h)']
X = X.values
```

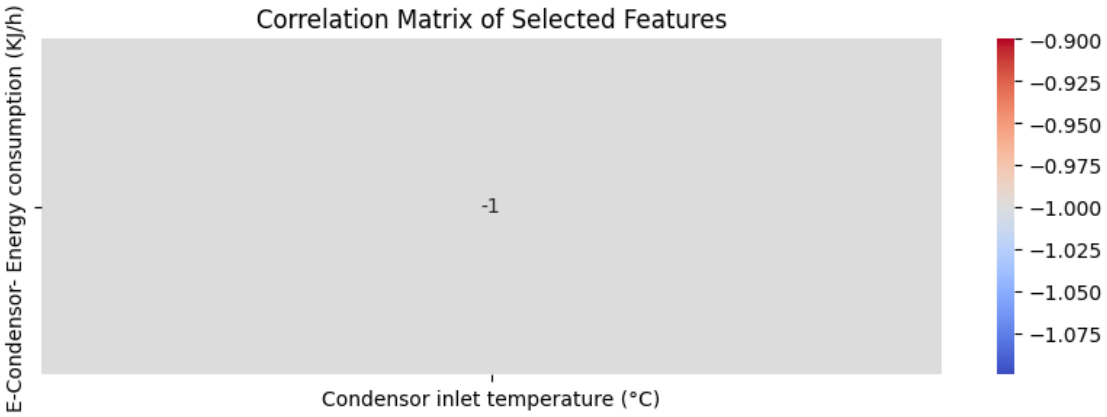
```
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:-2318202.284356667, c:509146085.55179334
```



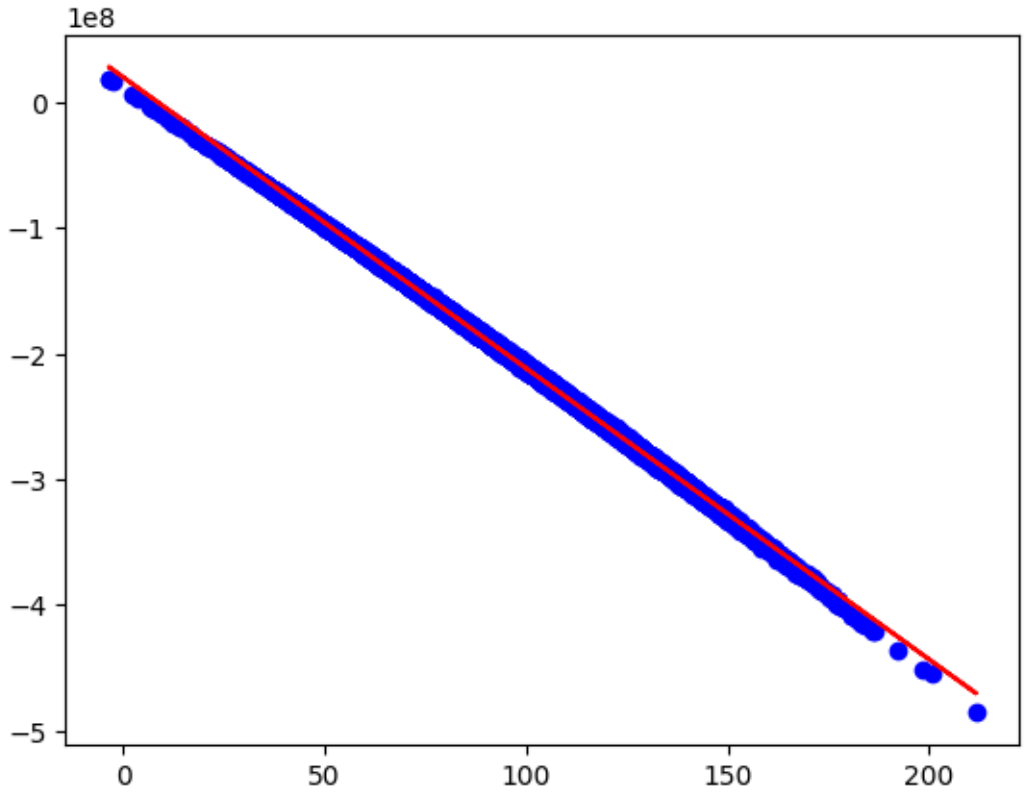
0.3.7 7. E-Condensor- Energy consumption

[47]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('E-Condensor- Energy consumption (KJ/h)', df)
```



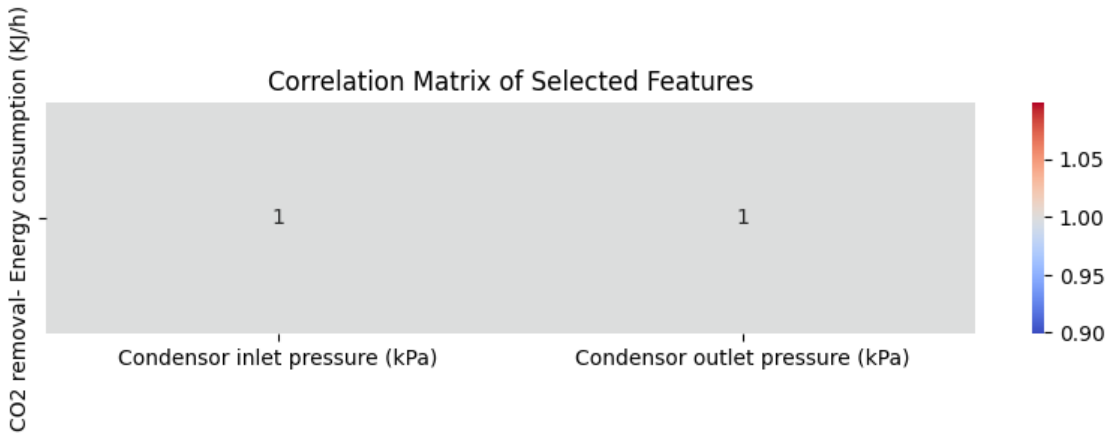
[48]:
```
X = df['Condensor inlet temperature (°C)']
y = df['E-Condensor- Energy consumption (KJ/h)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:-2318641.333250072, c:20190755.60255745
```

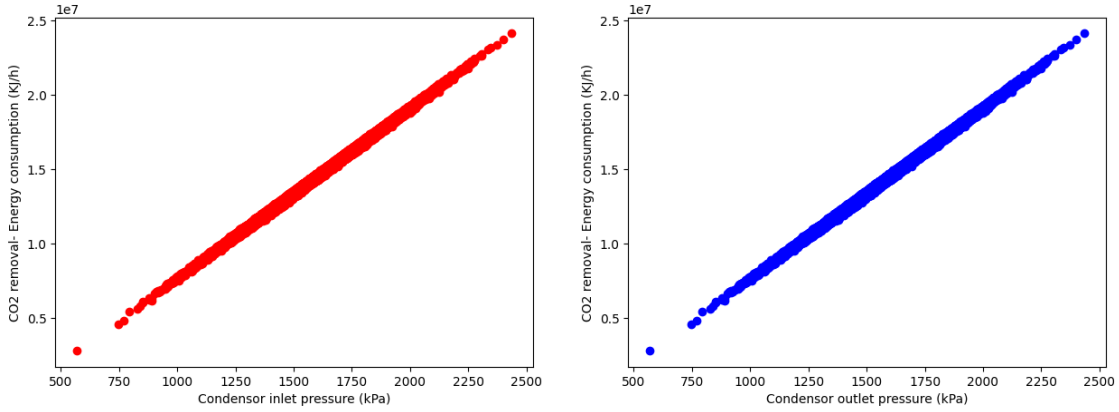


0.3.8 8. CO2 removal- Energy consumption

[50]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('CO2 removal- Energy consumption (KJ/h)', df)
```



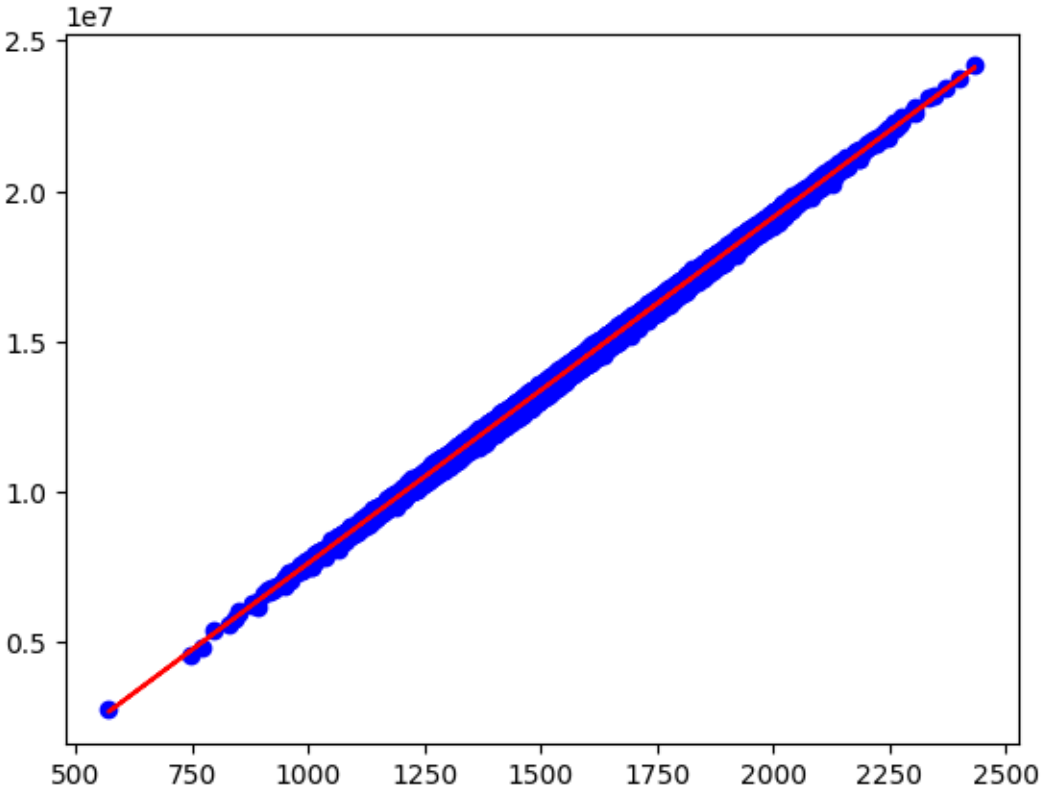
[51]:
```
#Plot the data to choose the model
X = df[['Condensor inlet pressure (kPa)','Condensor outlet pressure (kPa)']]
y = df['CO2 removal- Energy consumption (KJ/h)']
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].scatter(X['Condensor inlet pressure (kPa)'], y, color='red')
axes[0].set_xlabel("Condensor inlet pressure (kPa)")
axes[0].set_ylabel("CO2 removal- Energy consumption (KJ/h)")

axes[1].scatter(X['Condensor outlet pressure (kPa)'], y, color='blue')
axes[1].set_xlabel("Condensor outlet pressure (kPa)")
axes[1].set_ylabel("CO2 removal- Energy consumption (KJ/h)")



plt.show()
```



[52]:
```
X = df['Condensor inlet pressure (kPa)']
y = df['CO2 removal- Energy consumption (KJ/h)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
```

```
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:11485.991953742923, c:-3851589.240937948
```



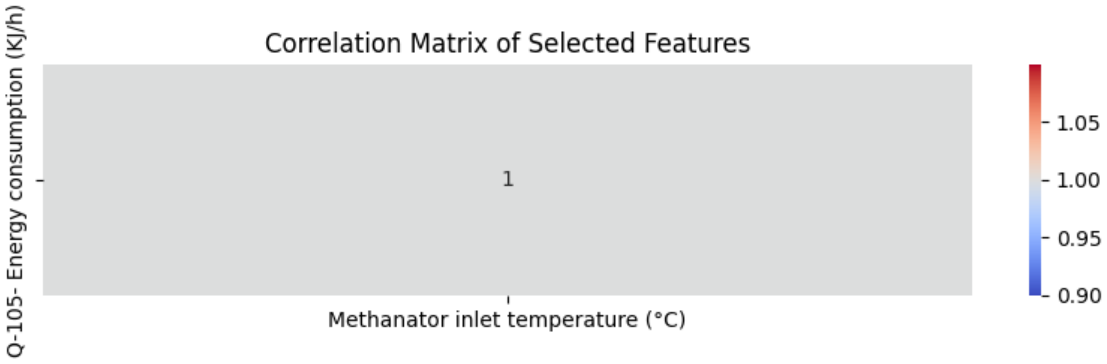
[53]:
```
print(df["Condensor inlet pressure (kPa)"]==df["Condensor outlet pressure␣
↪(kPa)"])
```

```
2         True
3         True
4         True
5         True
6         True
          ...
31998     True
31999     True
32000     True
32001     True
32002     True
Length: 32001, dtype: bool
```

The inlet and outlet condensor pressure are always the same , we can relay on the inlet to calculate the CO2 removal energy consumption.

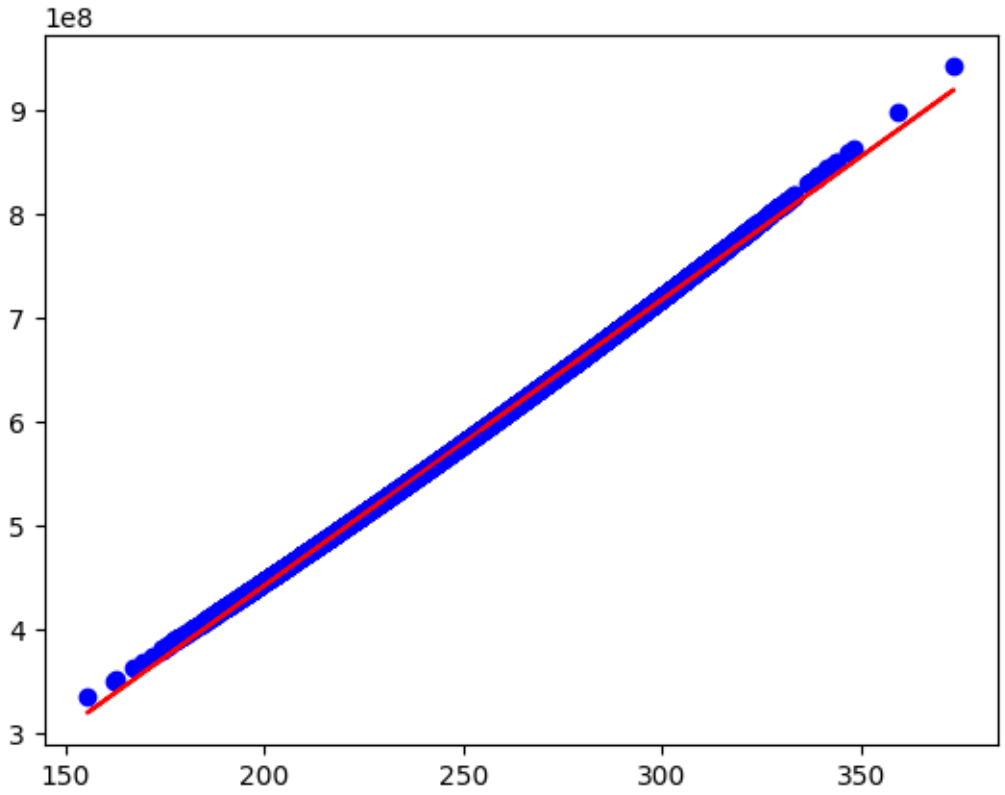
0.3.9 9. Q-105- Energy consumption

[54]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('Q-105- Energy consumption (KJ/h)', df)
```



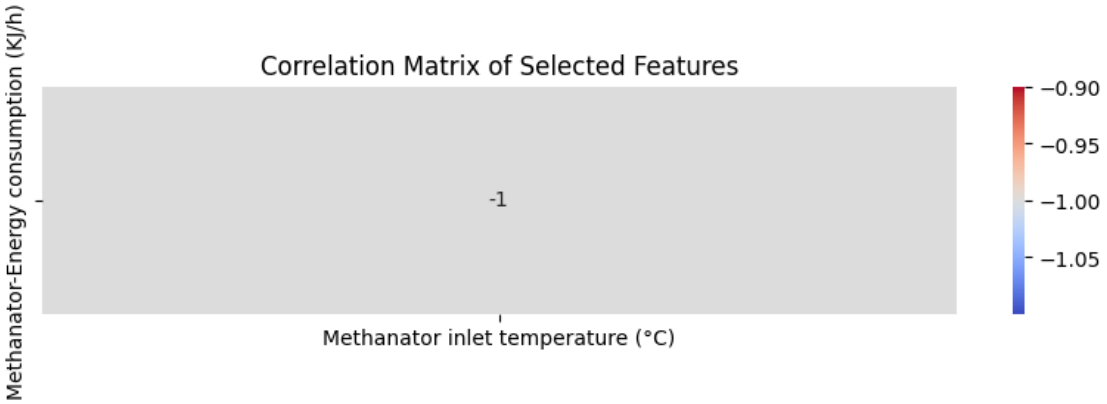
[55]:
```
X = df['Methanator inlet temperature (°C)']
y = df['Q-105- Energy consumption (KJ/h)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:2750563.303329907, c:-107470876.00437343
```



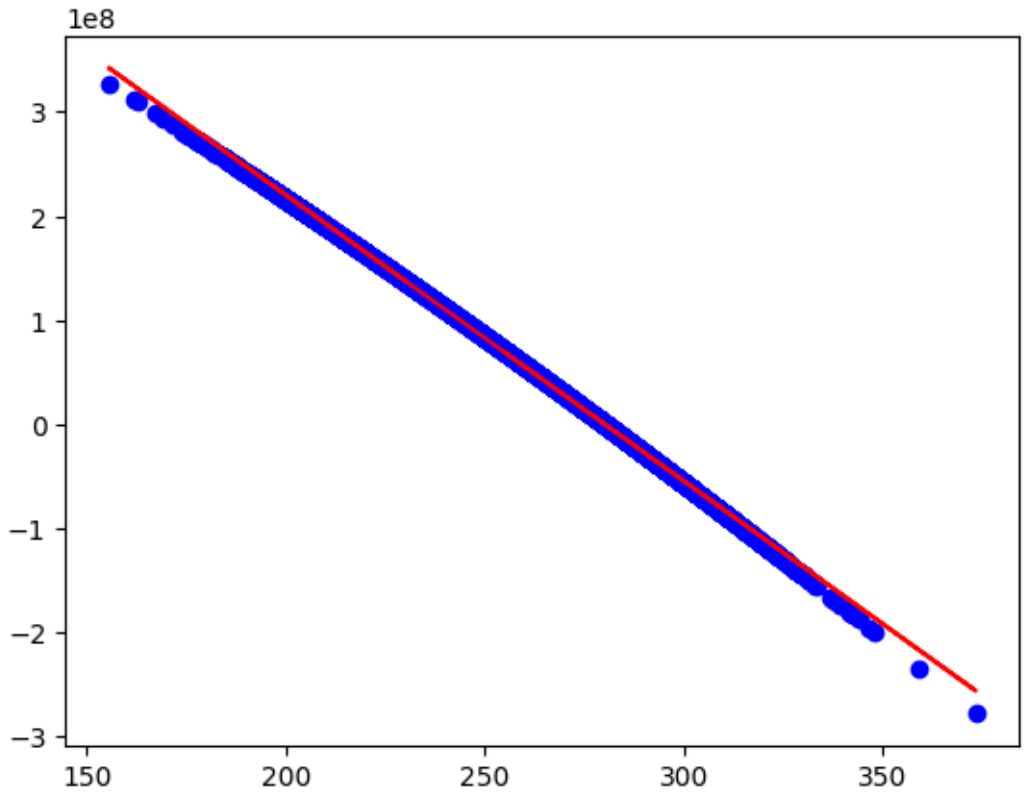
0.3.10 10. Methanator-Energy consumption

[57]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('Methanator-Energy consumption (KJ/h)', df)
```



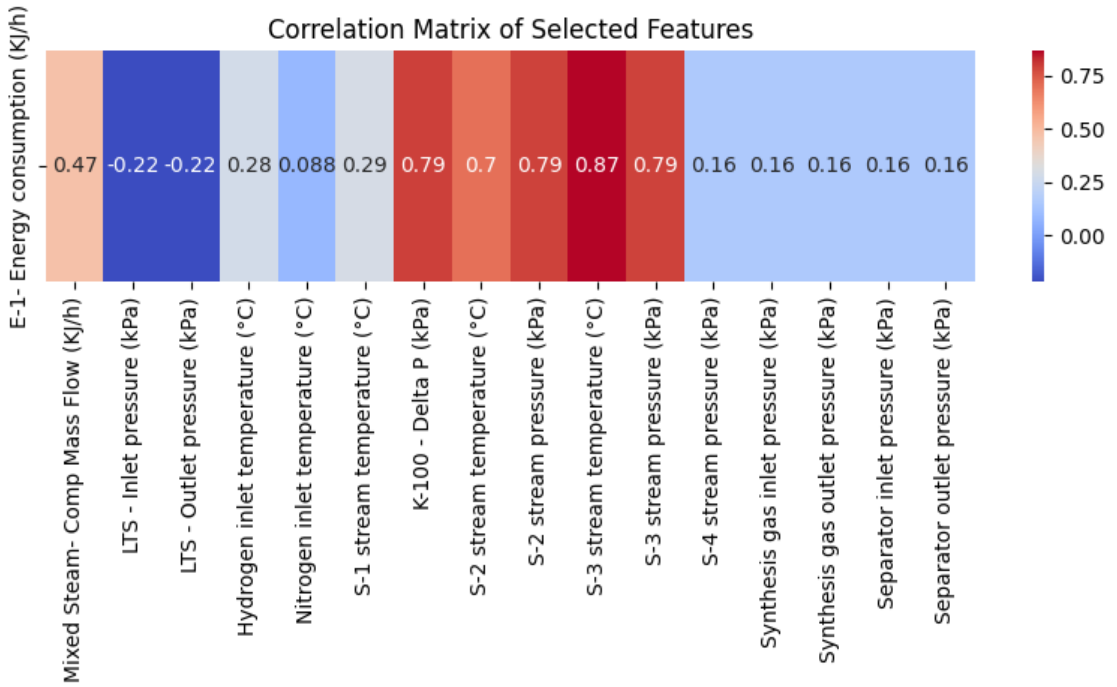
[58]:
```
X = df['Methanator inlet temperature (°C)']
y = df['Methanator-Energy consumption (KJ/h)']
X = X.values
y = y.values
mean_X = np.mean(X)
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:-2750561.059656047, c:770251086.4382389
```



0.3.11 11. E-1 Energy consumption

[50]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('E-1- Energy consumption (KJ/h)', df)
```

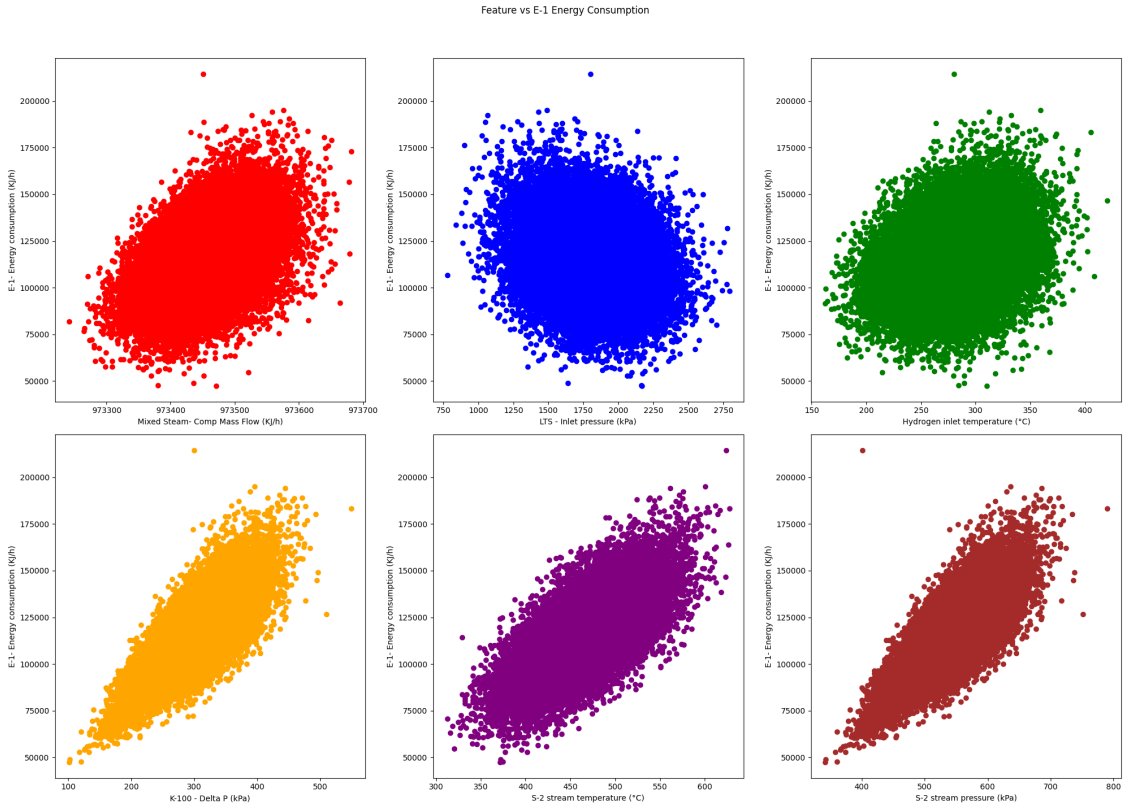


[102]:
```
X = df[['Mixed Steam- Comp Mass Flow (KJ/h)','LTS - Inlet pressure␣
↪(kPa)','Hydrogen inlet temperature (°C)',
       'K-100 - Delta P (kPa)', 'S-2 stream temperature (°C)']]
y = df['E-1- Energy consumption (KJ/h)']

mi_scores = make_mi_scores(X, y)
mi_scores
```

[102]:
```
K-100 - Delta P (kPa)                0.520937
S-2 stream temperature (°C)          0.334082
Mixed Steam- Comp Mass Flow (KJ/h)   0.128962
Hydrogen inlet temperature (°C)      0.042347
LTS - Inlet pressure (kPa)           0.029745
Name: MI Scores, dtype: float64
```

[61]:
```
fig, axes = plt.subplots(2, 3, figsize=(20, 15))
fig.suptitle('Feature vs E-1 Energy Consumption')


axes = axes.flatten()
```

```
colors = ['red', 'blue', 'green', 'orange', 'purple', 'brown', 'pink', 'gray',␣
 ↪'olive']

for idx, feature in enumerate(X.columns):
    axes[idx].scatter(X[feature], y, color=colors[idx])
    axes[idx].set_xlabel(feature)
    axes[idx].set_ylabel("E-1- Energy consumption (KJ/h)")

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



[103]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('RandomForest', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
```

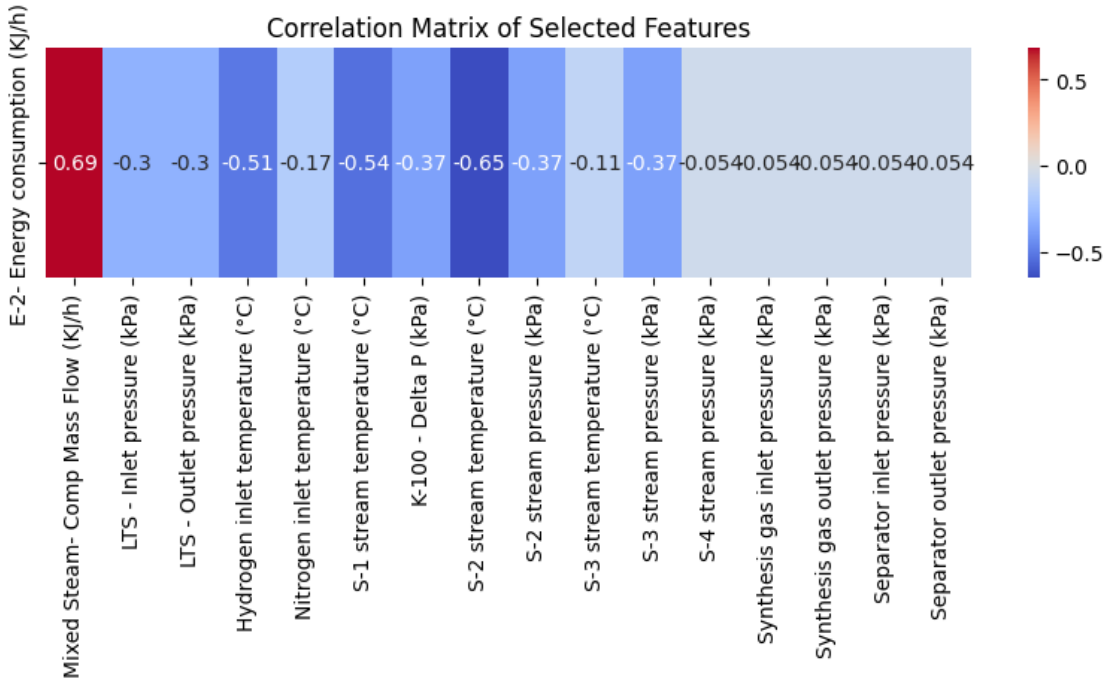
```
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/E-1-␣
 ↪Energy_Pipeline.pkl')
```

```
Mean Squared Error: 2928767.0020992514
R^2 Score: 0.9915310396721607
```

[103]:
```
['/home/houssam/internship project/trained_models/E-1- Energy_Pipeline.pkl']
```

0.3.12 12. E-2 Energy consumption

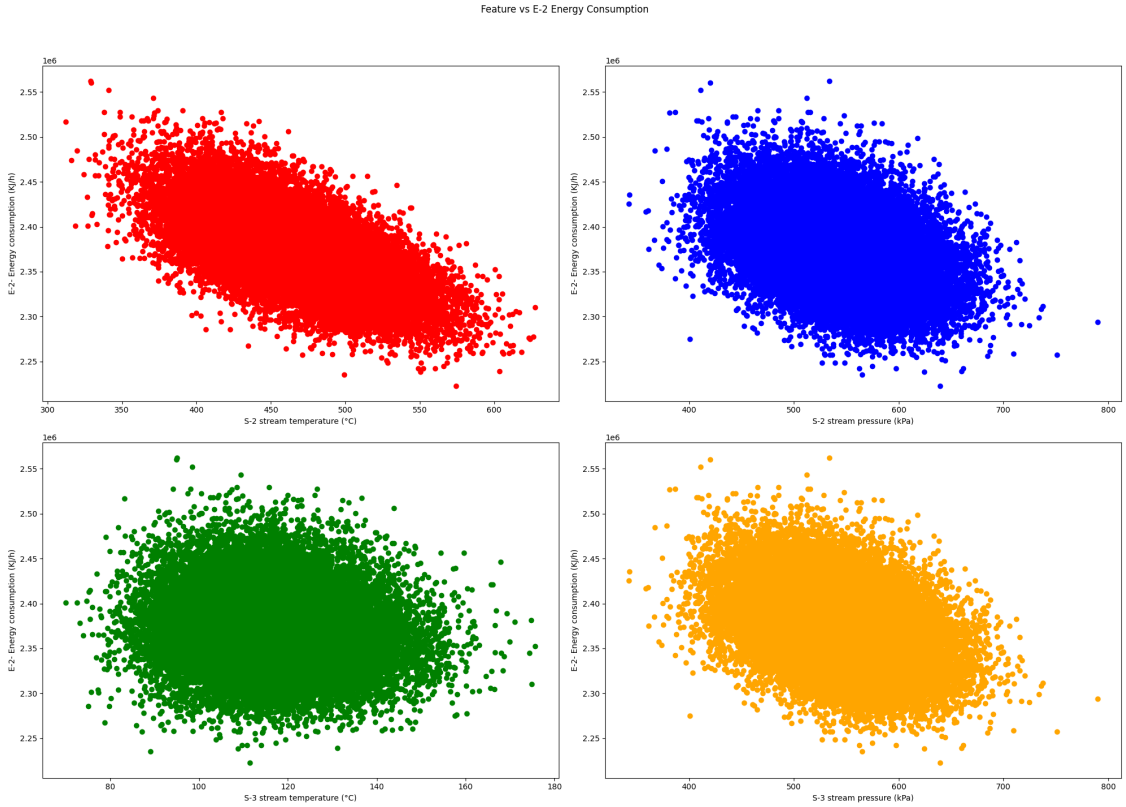
[19]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('E-2- Energy consumption (KJ/h)', df)
```



[104]:
```
X = df[['S-2 stream temperature (°C)','S-3 stream temperature (°C)','K-100 -␣
↪Delta P (kPa)']]
y = df['E-2- Energy consumption (KJ/h)']

mi_scores = make_mi_scores(X, y)
mi_scores
```

[104]:
```
S-2 stream temperature (°C)    0.274098
K-100 - Delta P (kPa)          0.078333
S-3 stream temperature (°C)    0.000389
Name: MI Scores, dtype: float64
```

[21]:
```
fig, axes = plt.subplots(2,2, figsize=(20, 15))
fig.suptitle('Feature vs E-2 Energy Consumption')


axes = axes.flatten()

colors = ['red', 'blue', 'green', 'orange']

for idx, feature in enumerate(X.columns):
    axes[idx].scatter(X[feature], y, color=colors[idx])
    axes[idx].set_xlabel(feature)
    axes[idx].set_ylabel("E-2- Energy consumption (KJ/h)")

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



[105]:
```
from sklearn.ensemble import RandomForestRegressor

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
  ↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('RandomForest', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
```

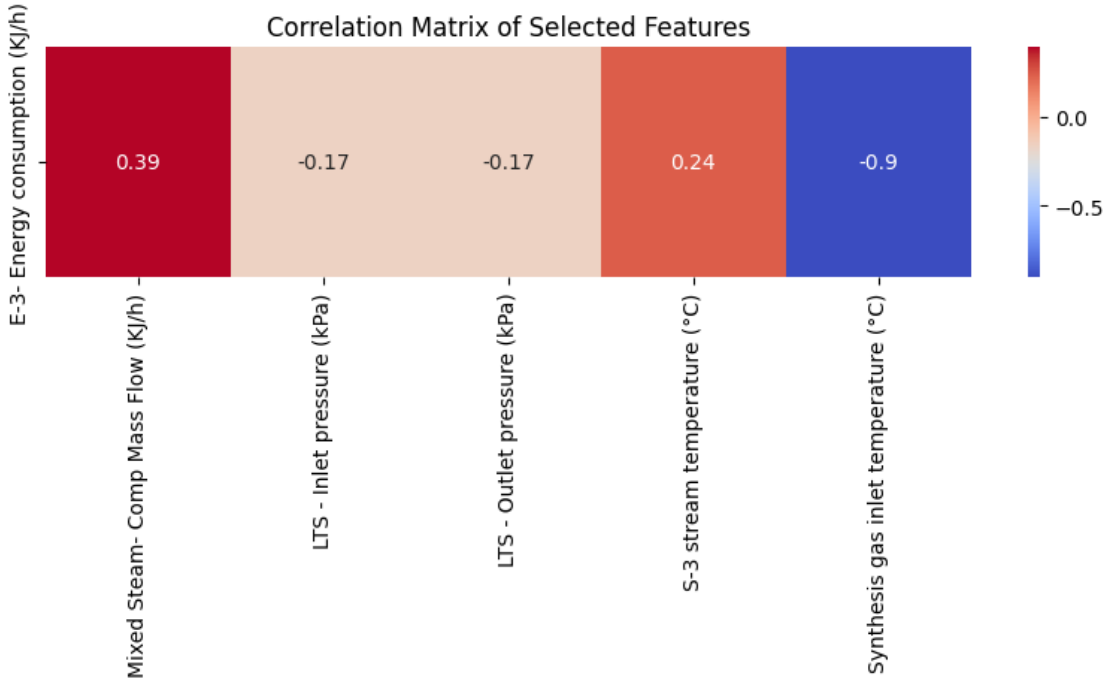
```
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/E-2-␣
↪Energy_Pipeline.pkl')
```

```
Mean Squared Error: 2144306.815090645
R^2 Score: 0.9986279126292663
```

[105]:
```
['/home/houssam/internship project/trained_models/E-2- Energy_Pipeline.pkl']
```

0.3.13 13. E-3 Energy consumption

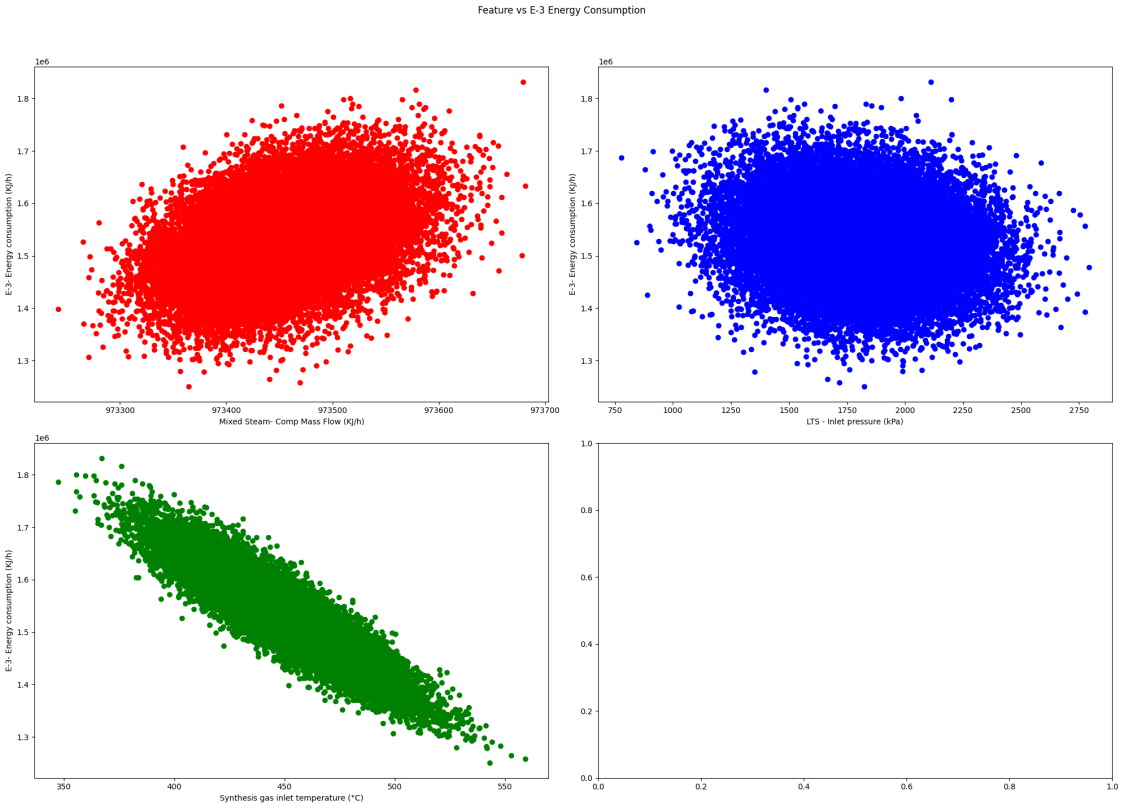
[68]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('E-3- Energy consumption (KJ/h)', df)
```



[77]:
```
X = df[['Mixed Steam- Comp Mass Flow (KJ/h)','LTS - Inlet pressure (kPa)',␣
↪'Synthesis gas inlet temperature (°C)']]
y = df['E-3- Energy consumption (KJ/h)']

mi_scores = make_mi_scores(X, y)
mi_scores
```

[77]:
```
Synthesis gas inlet temperature (°C)    0.848511
Mixed Steam- Comp Mass Flow (KJ/h)      0.079403
LTS - Inlet pressure (kPa)              0.017508
Name: MI Scores, dtype: float64
```

[70]:
```
fig, axes = plt.subplots(2,2, figsize=(20, 15))
fig.suptitle('Feature vs E-3 Energy Consumption')


axes = axes.flatten()

colors = ['red', 'blue', 'green', 'orange']

for idx, feature in enumerate(X.columns):
    axes[idx].scatter(X[feature], y, color=colors[idx])
    axes[idx].set_xlabel(feature)
    axes[idx].set_ylabel("E-3- Energy consumption (KJ/h)")

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



[78]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
```

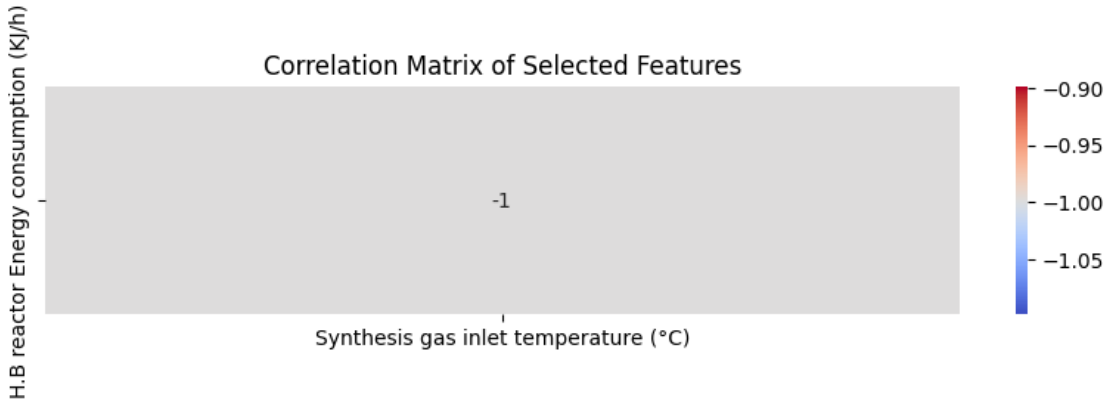
```
    ('RandomForest', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/E-3-␣
 ↪Energy_Pipeline.pkl')
```

```
Mean Squared Error: 16938777.704457786
R^2 Score: 0.9965815516518559
```

[78]:
```
['/home/houssam/internship project/trained_models/E-3- Energy_Pipeline.pkl']
```

0.3.14 14. H.B reactor Energy consumption

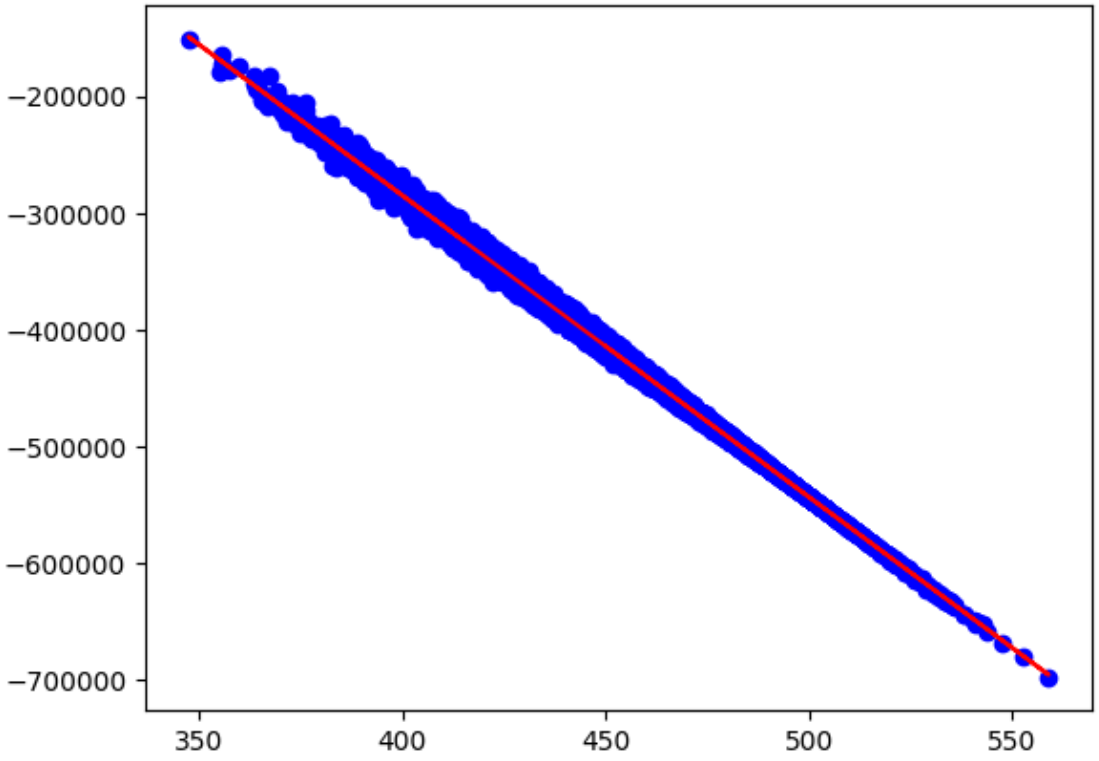
[73]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('H.B reactor Energy consumption (KJ/h)', df)
```



[74]:
```
X = df['Synthesis gas inlet temperature (°C)']
y = df['H.B reactor Energy consumption (KJ/h)']
X = X.values
y = y.values
mean_X = np.mean(X)
```

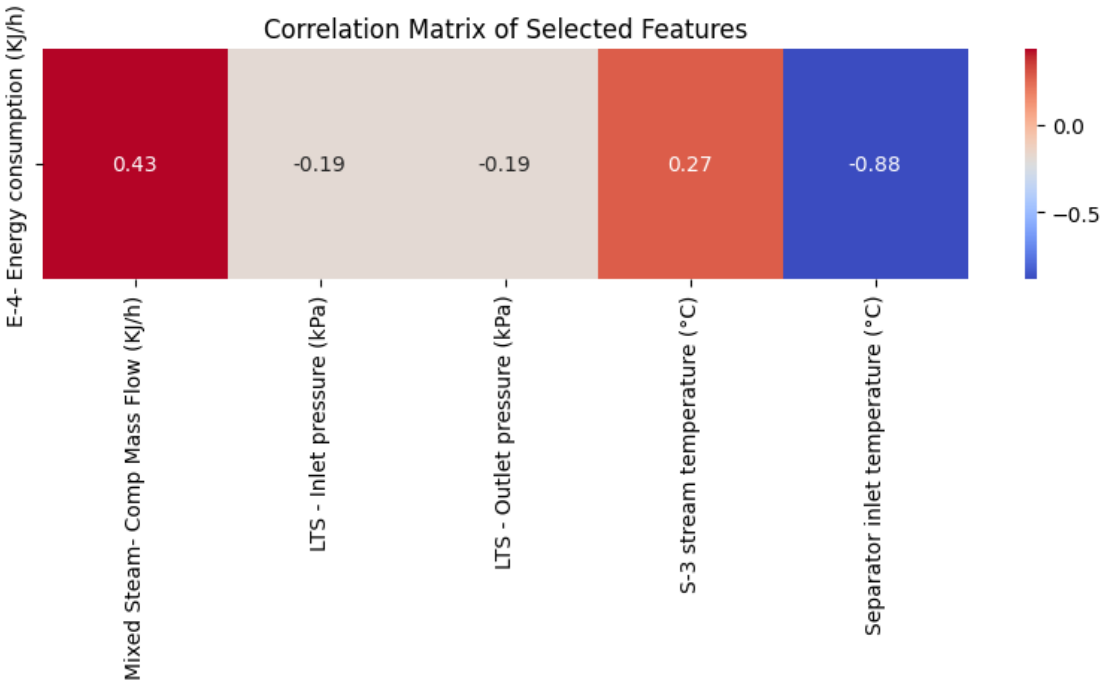
```
mean_y = np.mean(y)

# Calculate the slope (m)
m = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Calculate the intercept (c)
c = mean_y - m * mean_X

# Plot the data points and the regression line
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')

print(f"m:{m}, c:{c}")
```

```
m:-2579.93651444447, c:747019.723192719
```



0.3.15 15. E-4 Energy consumption

[75]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('E-4- Energy consumption (KJ/h)', df)
```

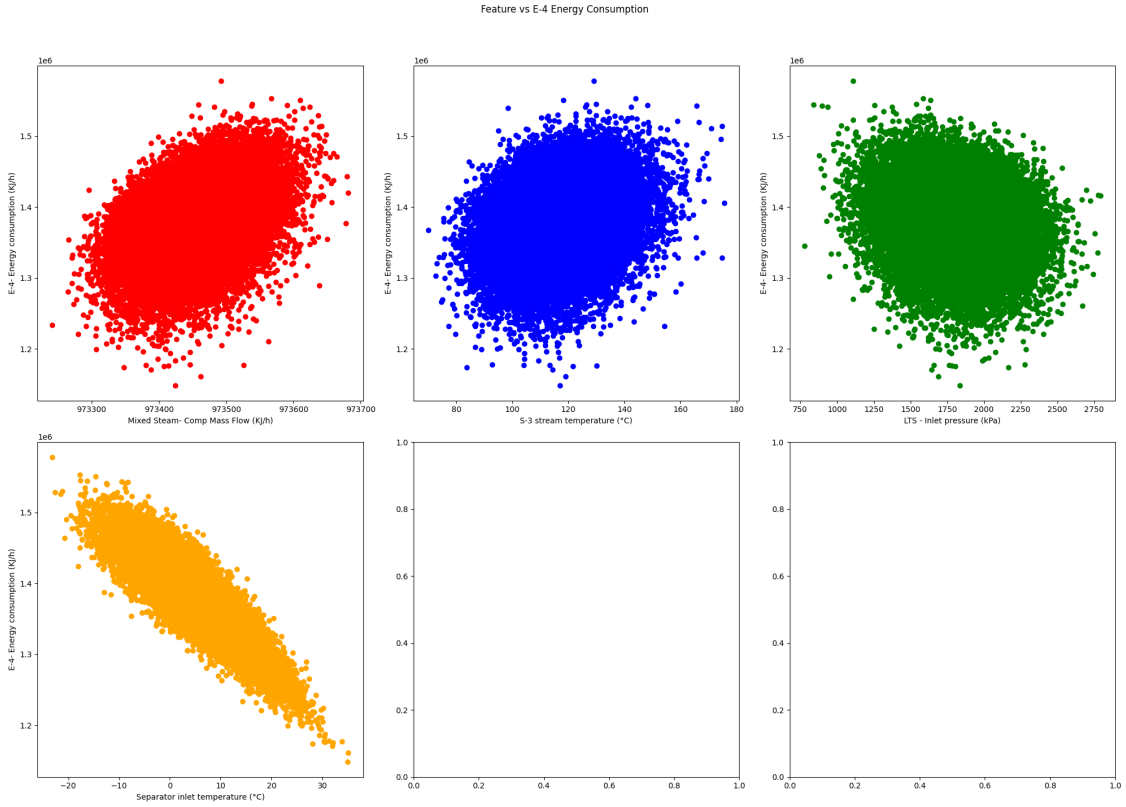


[106]:
```
X = df[['Mixed Steam- Comp Mass Flow (KJ/h)','LTS - Inlet pressure (kPa)',
        'Separator inlet temperature (°C)']]
y = df['E-4- Energy consumption (KJ/h)']

mi_scores = make_mi_scores(X, y)
mi_scores
```

[106]:
```
Separator inlet temperature (°C)      0.749652
Mixed Steam- Comp Mass Flow (KJ/h)    0.110885
LTS - Inlet pressure (kPa)            0.024353
Name: MI Scores, dtype: float64
```

[89]:
```
fig, axes = plt.subplots(2,3, figsize=(20, 15))
fig.suptitle('Feature vs E-4 Energy Consumption')


axes = axes.flatten()

colors = ['red', 'blue', 'green', 'orange','pink']

for idx, feature in enumerate(X.columns):
    axes[idx].scatter(X[feature], y, color=colors[idx])
    axes[idx].set_xlabel(feature)
    axes[idx].set_ylabel("E-4- Energy consumption (KJ/h)")
```

```
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



[107]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('RandomForest', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)


print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))
```

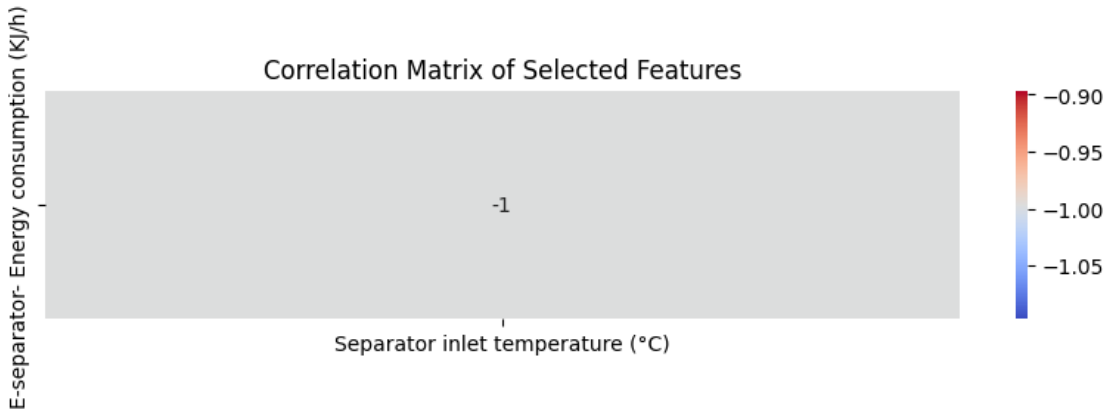
```
# Save the pipeline to a file
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/E-4-␣
↪Energy_Pipeline.pkl')
```

```
Mean Squared Error: 13155970.995607521
R^2 Score: 0.9944836645378353
```

[107]:
```
['/home/houssam/internship project/trained_models/E-4- Energy_Pipeline.pkl']
```

0.3.16 16. E-separator- Energy consumption

[99]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('E-separator- Energy consumption (KJ/h)', df)
```



[19]:
```
X = df[['Separator inlet temperature (°C)']]
y = df['E-separator- Energy consumption (KJ/h)']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('regression', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)
```

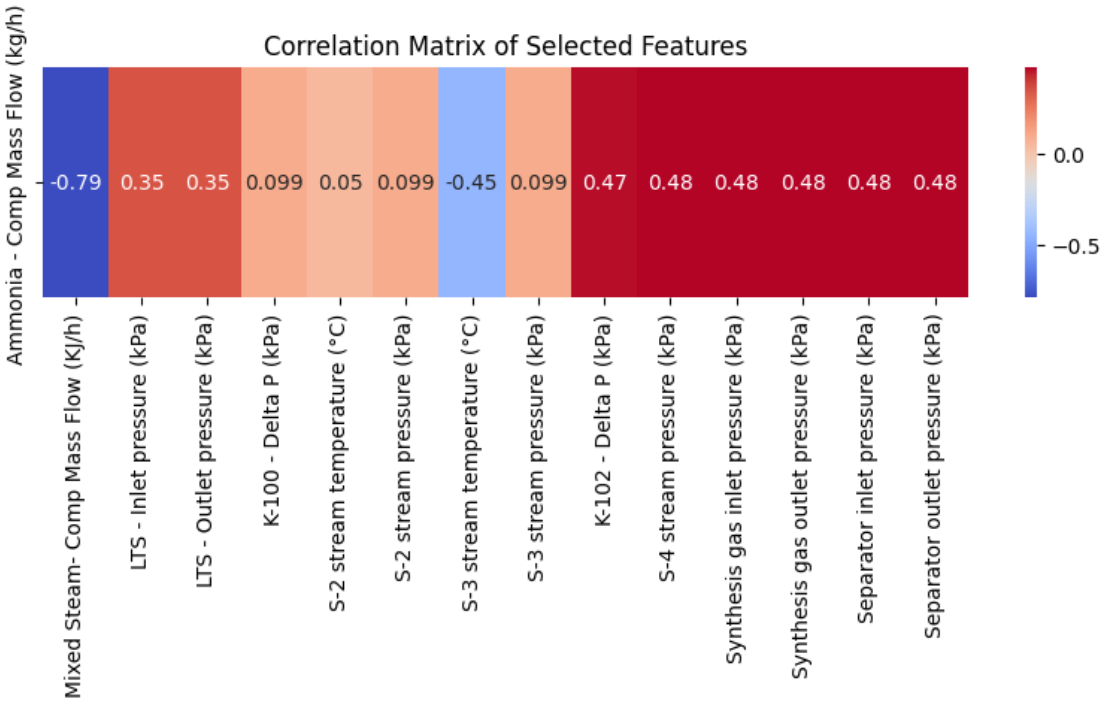
```
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/␣
 ↪E-separator- Energy_Pipeline.pkl')
```

```
Mean Squared Error: 1964962.420398348
R^2 Score: 0.998929434874485
```

[19]:
```
['/home/houssam/internship project/trained_models/ E-separator-
Energy_Pipeline.pkl']
```

0.4 Ammonia Output Mass Flow (kg/h) model

[101]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('Ammonia - Comp Mass Flow (kg/h)', df)
```

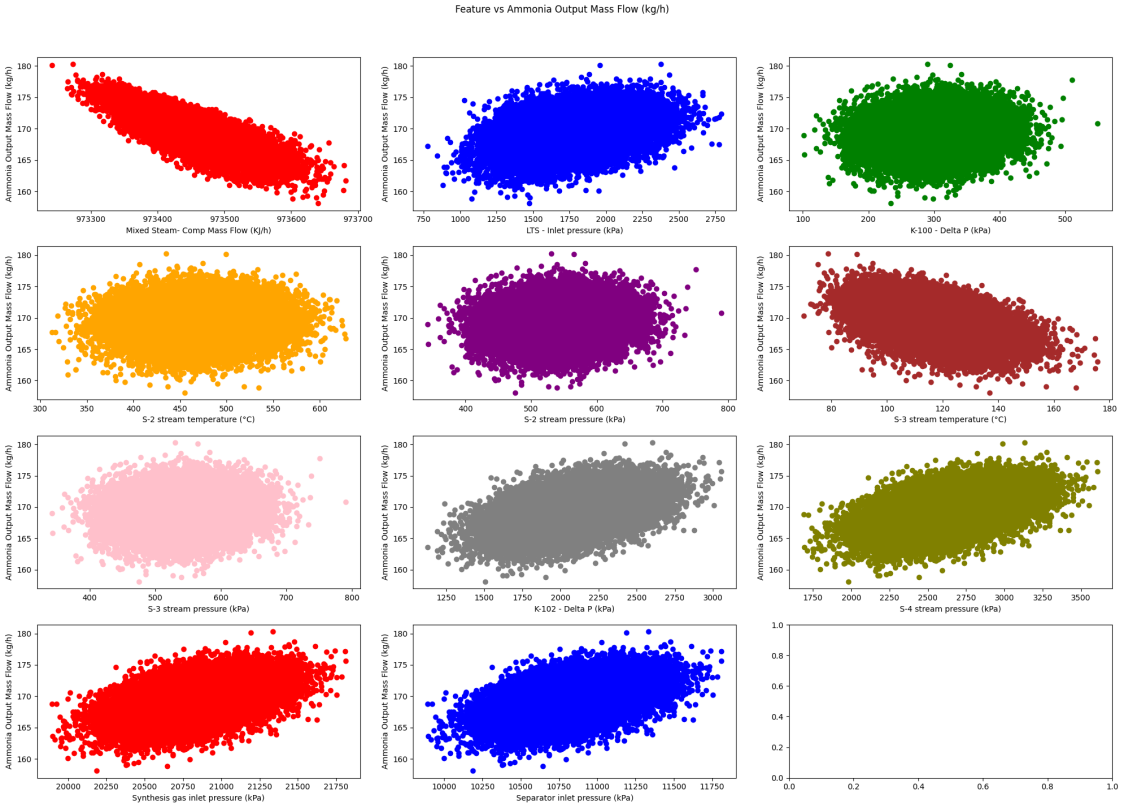


[108]:
```
X = df[['Mixed Steam- Comp Mass Flow (KJ/h)', 'LTS - Inlet pressure␣
 ↪(kPa)','K-100 - Delta P (kPa)', 'S-2 stream temperature (°C)','S-2 stream␣
 ↪pressure (kPa)', 'S-3 stream temperature (°C)',
```

```
       'S-3 stream pressure (kPa)', 'K-102 - Delta P (kPa)','S-4 stream␣
 ↪pressure (kPa)','Synthesis gas inlet pressure (kPa)','Separator inlet␣
 ↪pressure (kPa)']]
y = df['Ammonia - Comp Mass Flow (kg/h)']

mi_scores = make_mi_scores(X, y)
mi_scores
```

[108]:
```
Mixed Steam- Comp Mass Flow (KJ/h)    0.503501
Synthesis gas inlet pressure (kPa)    0.131096
S-4 stream pressure (kPa)             0.131095
Separator inlet pressure (kPa)        0.131094
K-102 - Delta P (kPa)                 0.128787
S-3 stream temperature (°C)           0.123006
LTS - Inlet pressure (kPa)            0.068686
K-100 - Delta P (kPa)                 0.010222
S-3 stream pressure (kPa)             0.010105
S-2 stream pressure (kPa)             0.010100
S-2 stream temperature (°C)           0.007301
Name: MI Scores, dtype: float64
```

[56]:
```
fig, axes = plt.subplots(4,3, figsize=(20, 15))
fig.suptitle('Feature vs Ammonia Output Mass Flow (kg/h)')


axes = axes.flatten()

colors = ['red', 'blue', 'green', 'orange', 'purple', 'brown', 'pink', 'gray',␣
 ↪'olive','red', 'blue']

for idx, feature in enumerate(X.columns):
    axes[idx].scatter(X[feature], y, color=colors[idx])
    axes[idx].set_xlabel(feature)
    axes[idx].set_ylabel("Ammonia Output Mass Flow (kg/h)")

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



[18]:
```
X = df[['Mixed Steam- Comp Mass Flow (KJ/h)', 'LTS - Inlet pressure (kPa)',␣
 ↪'S-2 stream temperature (°C)', 'S-3 stream temperature (°C)',
      'K-100 - Delta P (kPa)', 'K-102 - Delta P (kPa)','Synthesis gas inlet␣
 ↪pressure (kPa)','Separator inlet pressure (kPa)']]

y = df['Ammonia - Comp Mass Flow (kg/h)']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('regression', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

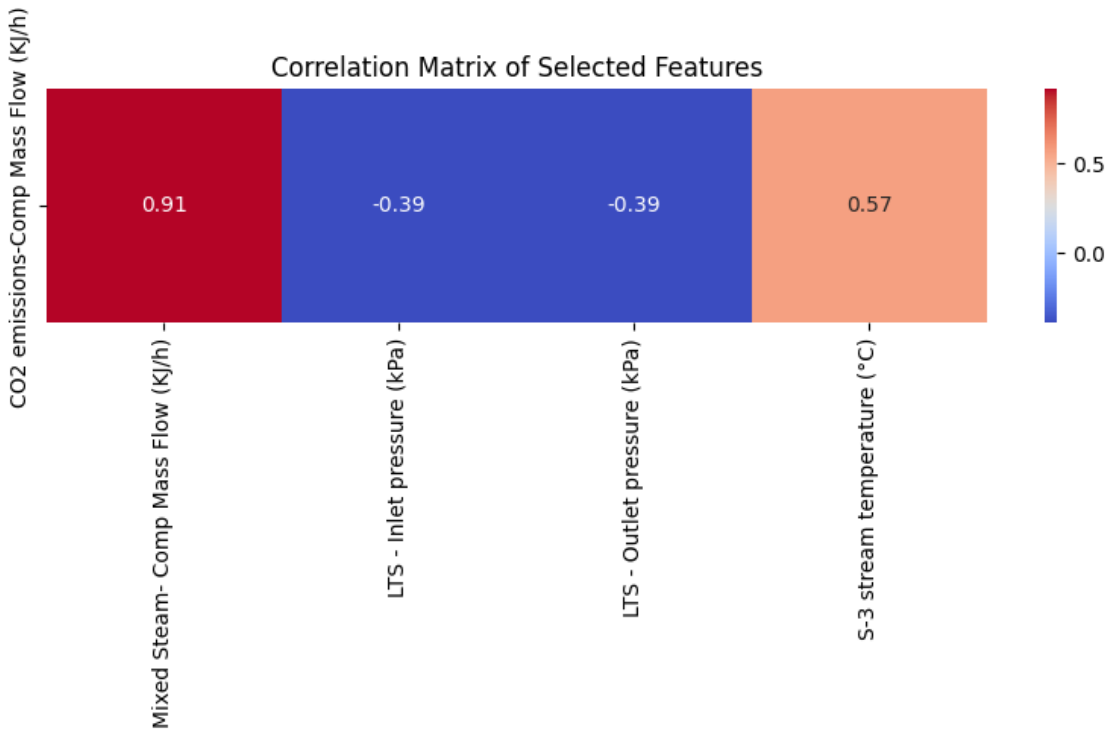
```
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
#joblib.dump(pipeline,'/content/drive/MyDrive/internship project/trained_models/
 ↪ Ammonia - Comp Mass Flow_Pipeline.pkl')
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/ Ammonia␣
 ↪- Comp Mass Flow_Pipeline.pkl')
```

```
Mean Squared Error: 0.07086740433664526
R^2 Score: 0.9886743331477847
```

[18]:
```
['/home/houssam/internship project/trained_models/ Ammonia - Comp Mass
Flow_Pipeline.pkl']
```

0.4.1 CO2 emissions-Comp Mass Flow

[105]:
```
#plot the sub_correlation_matrix
X = Sub_correlation_matrix('CO2 emissions-Comp Mass Flow (KJ/h)', df)
```

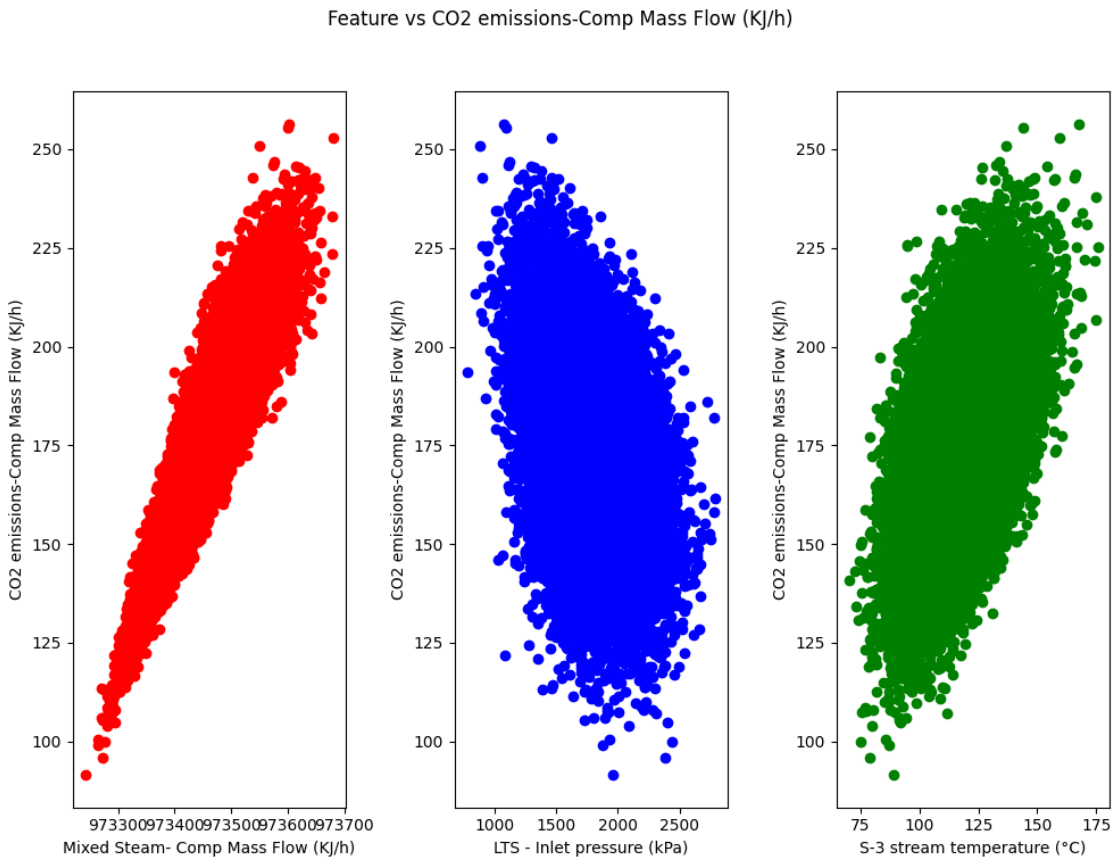


[111]:
```
X = df[['Mixed Steam- Comp Mass Flow (KJ/h)', 'LTS - Inlet pressure (kPa)']]
y = df['CO2 emissions-Comp Mass Flow (KJ/h)']
```

```
mi_scores = make_mi_scores(X, y)
mi_scores
```

[111]:
```
Mixed Steam- Comp Mass Flow (KJ/h)    0.944396
LTS - Inlet pressure (kPa)            0.079054
Name: MI Scores, dtype: float64
```

[112]:
```
fig, axes = plt.subplots(1,3, figsize=(10, 8))
fig.suptitle('Feature vs CO2 emissions-Comp Mass Flow (KJ/h)')


axes = axes.flatten()

colors = ['red', 'blue', 'green','pink']
for idx, feature in enumerate(X.columns):
    axes[idx].scatter(X[feature], y, color=colors[idx])
    axes[idx].set_xlabel(feature)
    axes[idx].set_ylabel("CO2 emissions-Comp Mass Flow (KJ/h)")

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



[112]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

pipeline = Pipeline([
    ('scaler', StandardScaler()),           # Normalize the input columns
    ('regression', RandomForestRegressor()) # Train a RandomForestClassifier
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Test the pipeline
y_pred = pipeline.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))


# Save the pipeline to a file
#joblib.dump(pipeline,'/content/drive/MyDrive/internship project/trained_models/
↪ Ammonia - Comp Mass Flow_Pipeline.pkl')
joblib.dump(pipeline,'/home/houssam/internship project/trained_models/CO2␣
↪emissions-Comp Mass Flow_Pipeline.pkl')
```

```
Mean Squared Error: 3.820423632597303
R^2 Score: 0.9898589886554833
```

[112]:
```
['/home/houssam/internship project/trained_models/CO2 emissions-Comp Mass
Flow_Pipeline.pkl']
```